

Formation

Tests webservices avec Postman

Décembre 2022

onepoint.

Programme

1. Introduction aux tests API
2. Installation et configuration Postman
3. Création des requêtes simples
4. Ajout des vérifications
5. Enchaînement de plusieurs requêtes
6. Utilisation des variables
7. Utilisation des données CSV / JSON
8. Création d'une boucle
9. API Mock
10. Documentation des APIs (swagger)
11. Annexes



1

Introduction aux tests API

Rappel ISTQB : niveaux de test

Le syllabus de l'ISTQB définit 4 niveaux de test principaux :

1. Les tests composants : code fonctionne correctement de manière isolée
2. Les tests d'intégration : les différents éléments fonctionnent ensemble
3. Les tests système : le système complet fonctionne correctement
4. Les tests d'acceptation : répond aux attentes des utilisateurs

Conseil :

Penser à les différencier par les vérifications qui sont faites dans chaque niveau, et les risques couverts.

Rappel ISTQB : techniques de test

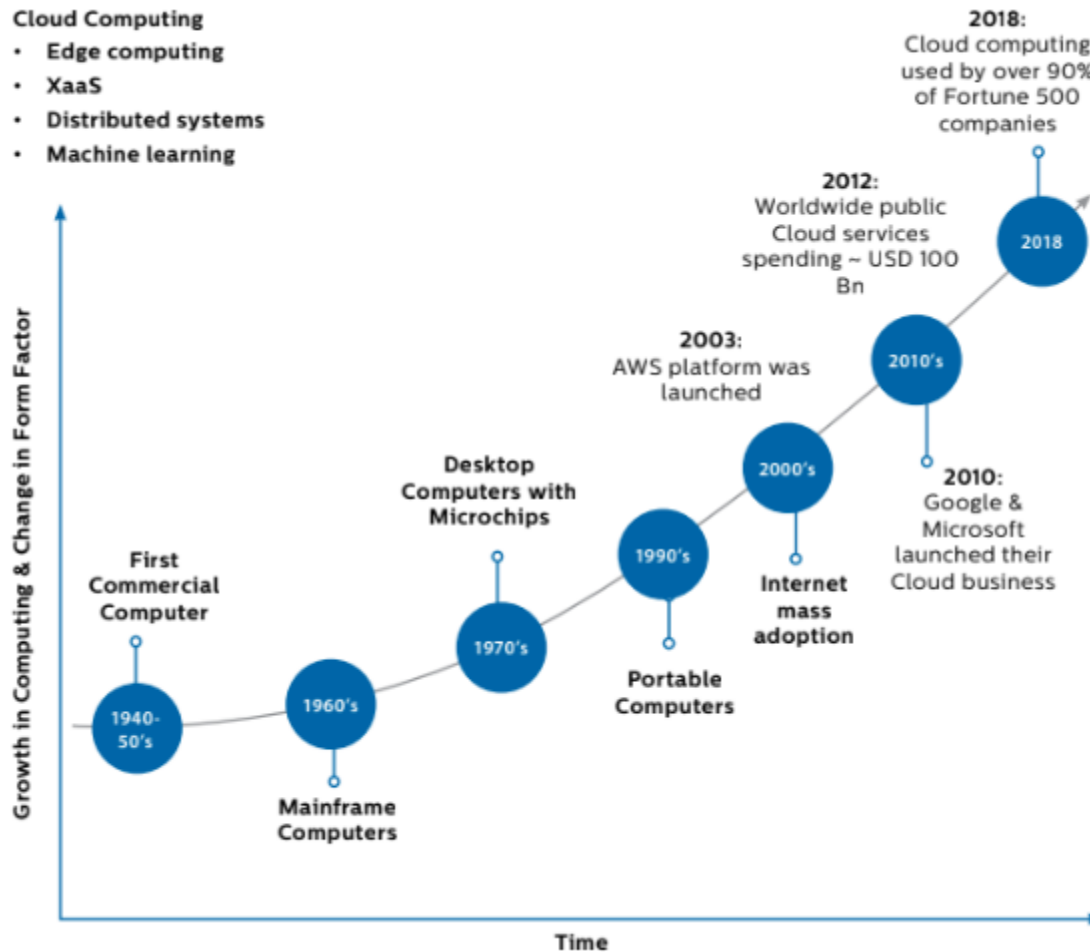
- Boîte noire
 1. Partitions d'équivalence : grouper les possibilités similaires. 1 représentant de chaque groupe pour voir si ça marche.
 2. Analyse des valeurs limites : exacte, un peu moins, un peu plus
 3. Test de tables de décision : toutes les combinaisons possibles
 4. Test des transitions d'état : à travers ses différents états
 5. Test des cas d'utilisation : se mettre à la place de l'utilisateur
- Boîte blanche
 1. Test et couverture des instructions : chaque ligne de code est exécutée au moins une fois lors des tests.
 2. Test et couverture des décisions : chaque condition (true/false) dans le code est testée au moins une fois.

Remarque : Pour les tests API, on utilisera couramment les tests en boîte grise.

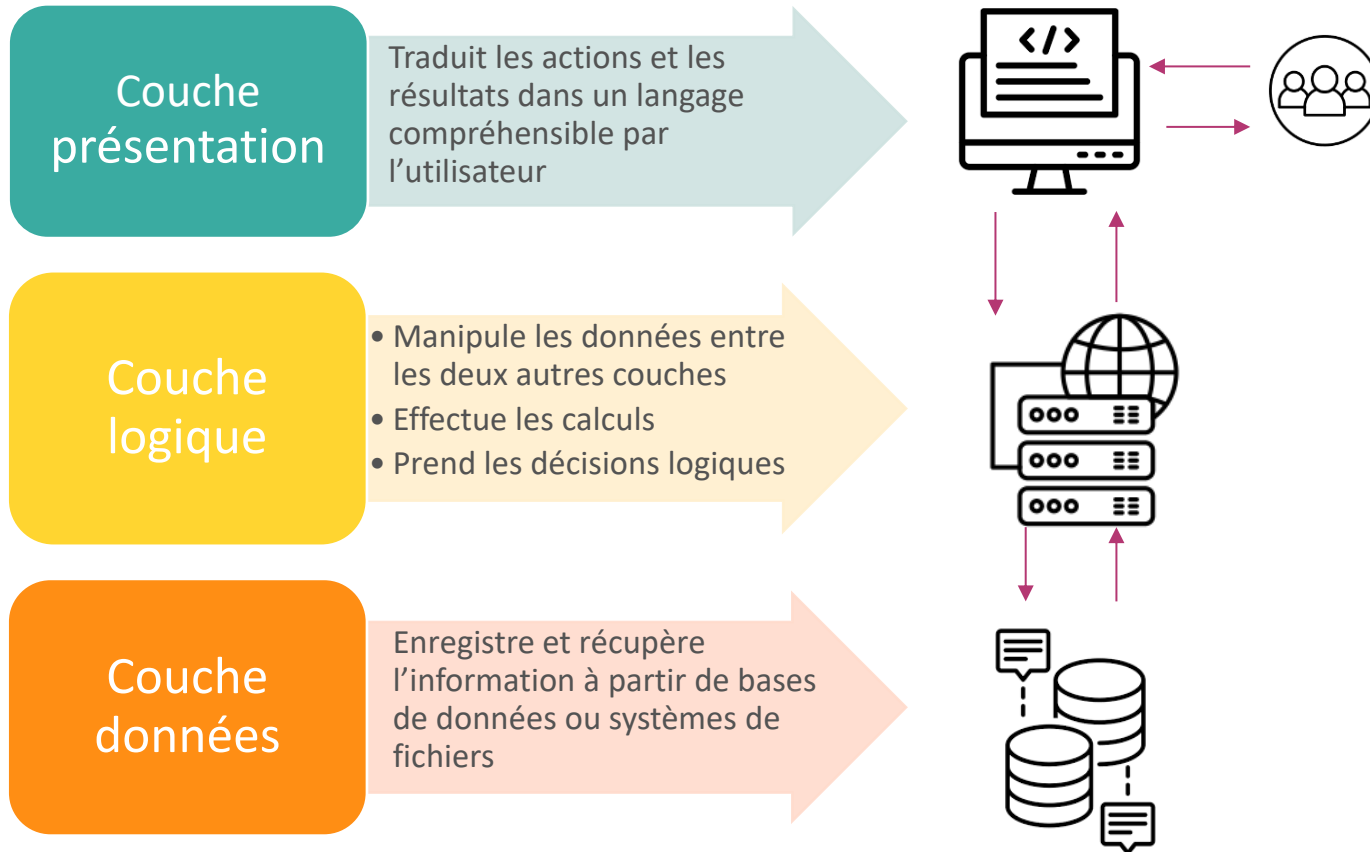
Rappel ISTQB : Définition d'un cas de test

Cas de test : un ensemble de valeurs d'entrée, de **préconditions** d'exécution, de **résultats attendus** et de post conditions d'exécution, développées pour un objectif ou une condition de tests particulier, tel qu'exécuter un chemin particulier d'un programme ou vérifier le respect d'une **exigence spécifique** [d'après IEEE 610]

Evolution des architectures



Architecture n-tiers



Les API – définition

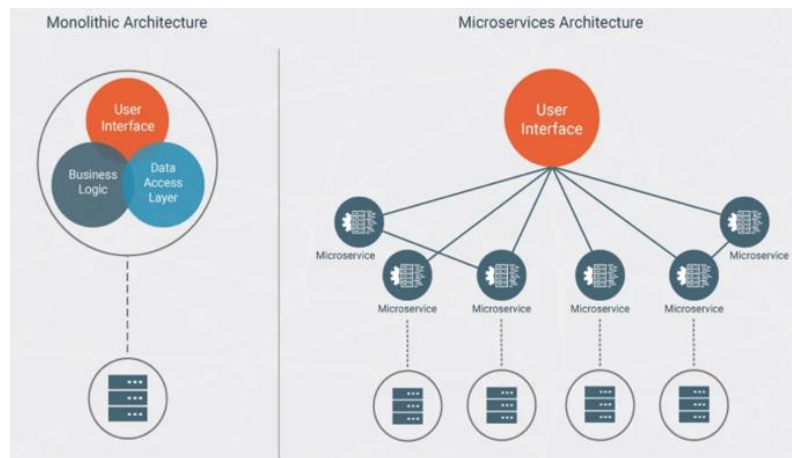
- Les API (Application Programmable Interface) sont des ensembles de règles et standards qui étendent les fonctionnalités d'un composant ou d'une application.
- Ce sont des interfaces standardisées qui permettent à plusieurs applications d'échanger des informations.
- Une API définit en général :
 - Les types de messages et de données que le composant peut accepter en entrée
 - Les types et les contenus des réponses
- Les composants peuvent appartenir à la même application ou être fournis par des services tiers

Un web service est une implémentation Web d'une API.



Pourquoi les API ?

- Les applications développées doivent être flexibles et pouvoir s'adapter à différents environnements
- Elles doivent souvent communiquer avec d'autres applications ou services du SI et doivent s'interfacer facilement
- Elles devront en outre pouvoir intégrer des changements dans un minimum de temps
- Cela s'appuie nécessairement sur une architecture applicative construite sur des composants suffisamment petits et réutilisables
- Les assemblages de composants peuvent varier en fonction des besoins
- Les interfaces entre ces composants sont donc essentielles au bon fonctionnement du système complet

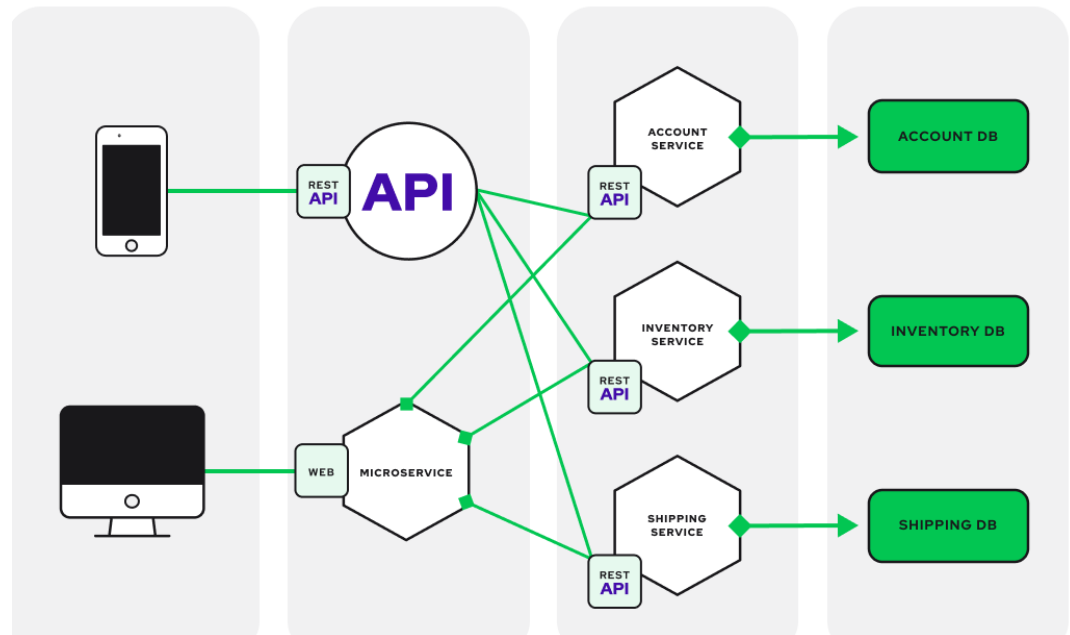


Architecture microservices

- Approche architecture et organisation du développement logiciel.
- Découpage en petits services indépendants qui communiquent via des API bien définies.
- Simplifient la mise à l'échelle et accélèrent le développement des applications
- Caractéristiques : Autonomie et spécialisation

Avantages

- Agilité
- Flexibilité pour le dimensionnement
- Facilité de déploiement
- Indépendance de la technologie
- Réutilisabilité du code
- Résilience



Quelques exemples d'API

- Paiement sécurisé en ligne



Verified by
VISA

Identification

Pour vous protéger contre l'utilisation frauduleuse de votre carte bancaire, la vous demande de vous identifier en saisissant votre date de naissance.

Marchand : J'achète.fr
Montant : 75€
Date : 10/06/2009
N° de carte : xxxx xxxx xxxx 1234

Date de naissance
du porteur de la carte : / /

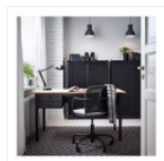
jj mm aaaa

Cette identification est obligatoire pour conclure votre transaction. Si vous refusez de vous identifier, votre achat sera annulé.

► [Ne pas m'identifier et annuler mon achat](#)

- Calculs de temps de trajet et tarif en fonction de la localisation

- Disponibilité d'articles en magasin



>

Disponibilités en magasin

Magasin: Avignon ▼

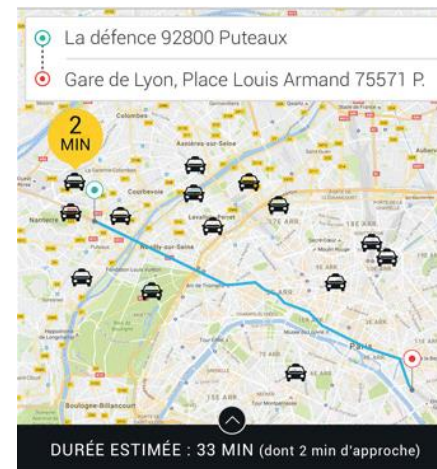


Disponible à
Avignon

[Voir les prévisions](#)

Disponible en ligne

Disponible pour l'achat en ligne

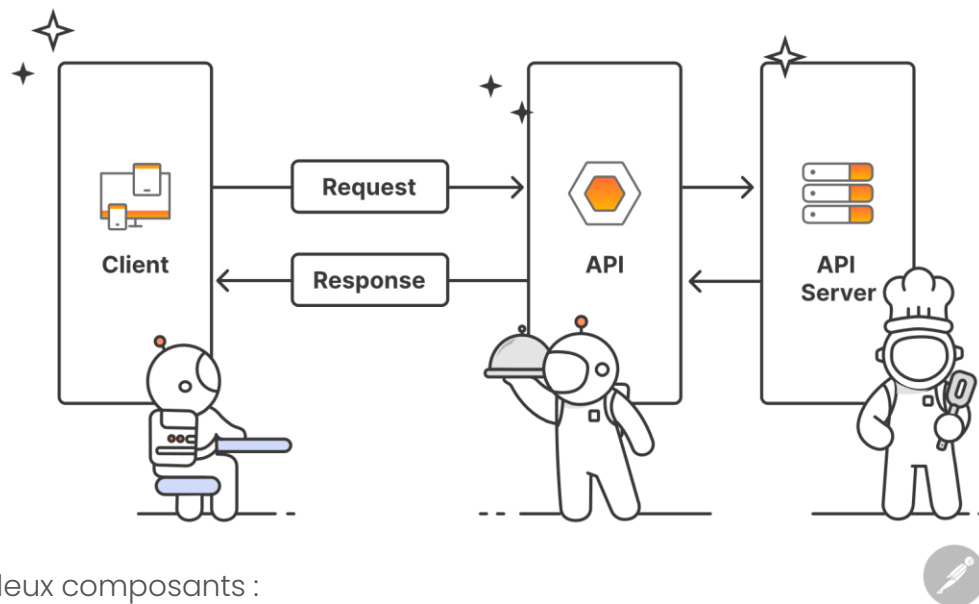


37 - 46€

onepoint.

Fonctionnement des API

Le concept fondamental d'une API web est de permettre l'échange de données en utilisant les protocoles HTTP/HTTPS

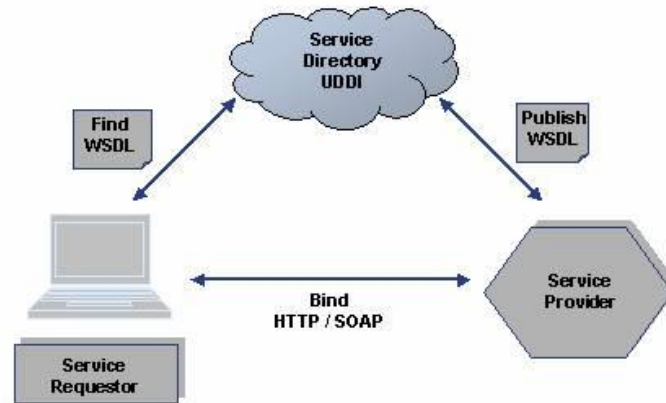


Il s'appuie sur deux composants :

- Structure des données : format spécifique utilisé par les requêtes et réponses API (XML, JSON...)
- Transfert et sécurisation des données : définit la façon dont les données sont échangées et manipulées en fonction des méthodes HTTP utilisées (PUT, POST, GET...) et des protocoles de Web Services utilisés comme SOAP ou REST

Web Services

- Un web service est une méthode de communication entre deux dispositifs connectés sur le web. C'est un service logiciel fourni à une adresse réseau (endpoint) et qui est actif en permanence (always on)



- Le langage WSDL (Web Services Description Language) est un langage de description d'interface au format XML qui décrit la fonctionnalité offerte par un Web Service.
- Le fichier WSDL Associé au Web Service définit :
 - La manière dont le service peut être appelé
 - Les paramètres qu'il attend
 - La structure de la réponse qu'il fournit

Web Services

Les web services reposent sur différents protocoles et technologies pour fonctionner

Protocoles

- **HTTP/HTTPS** : protocole de communication client-serveur développé pour le World Wide Web. Il est utilisé pour échanger toute sorte de données entre client HTTP et serveur HTTP.

Architecture

- **REST** : style d'architecture pour les environnements distribués décrivant comment construire des applications sur internet.
- **SOAP** : protocole standard de transmission de message décrit en XML. Il se présente comme une enveloppe pouvant être signée et pouvant contenir des données ou des pièces jointes. Il utilise le protocole HTTP et permet d'effectuer des appels de méthodes à distance.

Formats

- **XML** : Langage de balisage générique. On dit que ce langage est « extensible » car il permet de définir des espaces de noms, c'est-à-dire des langages ayant chacun leur propre vocabulaire et leur propre grammaire.
- **JSON** : JavaScript Objet Notation est un langage léger d'échange de données textuelles avec une syntaxe simple et une structure en arborescence
- **HTML** : Langage de balisage pour représenter les pages web.
- **WSDL** : Langage de description des Web services. Il s'agit de l'interface présentée aux utilisateurs qui indique comment utiliser un web service (types des données échangées, messages, etc.).

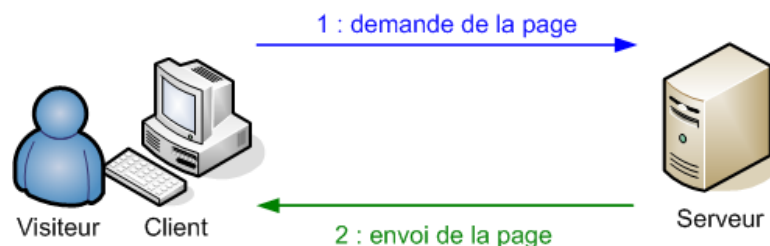
Langages

- **XPATH** : Langage permettant de requêter des portions dans un document XML.
- **XQUERY** : Langage permettant à la fois d'extraire des informations d'un document XML mais également de réaliser des calculs sur ces dernières.
- **JSONPATH** : syntaxe d'extraction des données JSON

Web Services

Protocole HTTP et requêtes

- HTTP est un protocole permettant la communication entre un client et un serveur. Une requête est toujours composée d'un appel (du client) et d'une réponse (du serveur). Voici un schéma résumant l'échange, les requêtes sont symbolisées par les flèches.



- Syntaxe d'une requête et d'un réponse HTTP:

Requête Client	Réponse Serveur
Ligne de commande <u>Syntaxe</u> : Commande, URL, version de protocole. <u>Exemple</u> : <code>GET /repertoire/page.html HTTP/1.1</code>	Ligne de statut <u>Syntaxe</u> : version, Code-réponse, Texte-réponse. <u>Exemple</u> : <code>HTTP/1.1 200 OK</code>
En-têtes de la requête <u>Syntaxe</u> : « Nom entête :valeur ». <u>Exemple</u> : <code>Host: www.site.com</code>	En-têtes de réponse. <u>Syntaxe</u> : « Nom entête :valeur ». <u>Exemple</u> : <code>Server: Apache/2.2.3</code>
Saut de ligne	Saut de ligne
Corps de la requête sert principalement à transmettre les formulaires HTML.	Corps de réponse Contient le contenu, dans le cas d'une page web le HTML par exemple.

SOAP vs REST

SOAP

→ Orienté Services

Avantages

- Mécanisme d'appel de procédures distantes, on peut s'appuyer sur Soap dans un cadre contractuel.
- Supporte le transactionnel
- Formel, les méthodes et résultats sont spécifiés via des contrats (WSDL).
- Fiable, mécanismes garantissant la fiabilité de la transmission et réception des données.
- Soap est intégré dans la plupart des environnements de développement

Inconvénients

- Soap est considéré comme trop complexe et verbeux pour des besoins ordinaires.
- Nécessite des outils pour le développement

REST

→ Orienté Ressources (pages, images, etc.)

Avantages

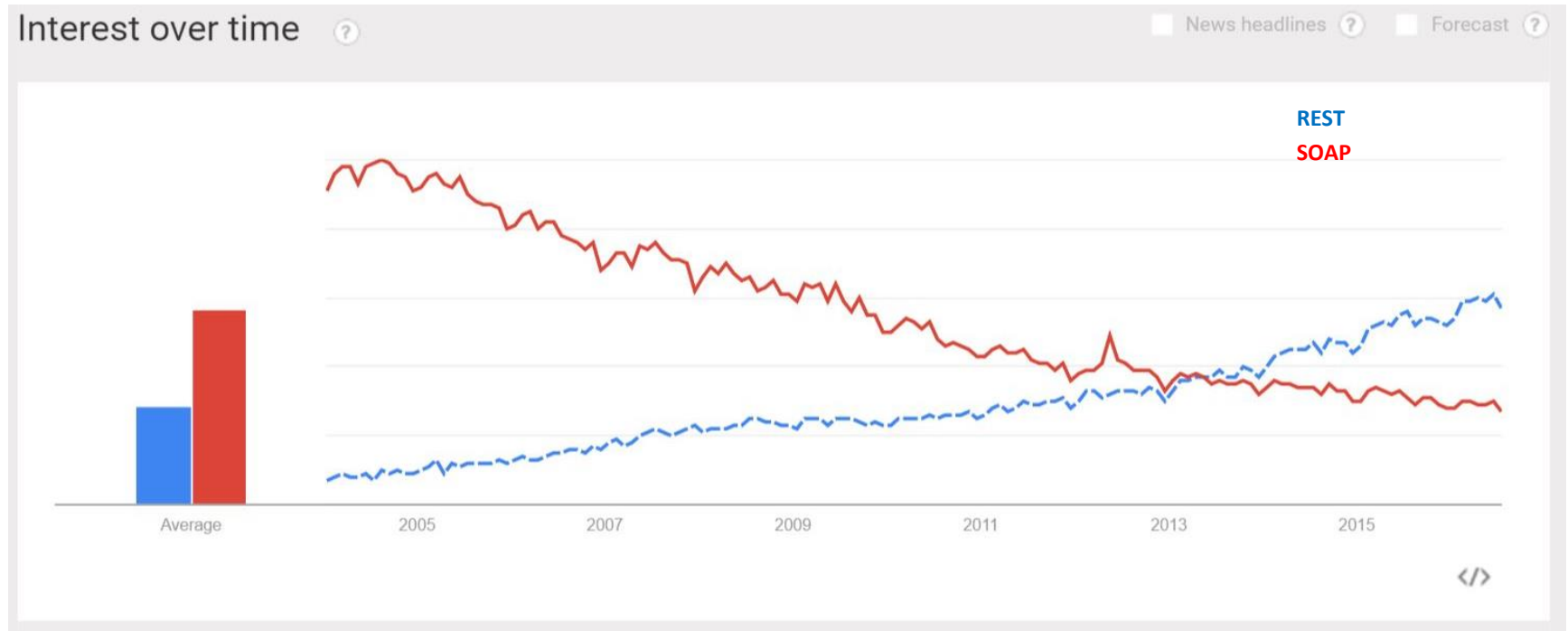
- Léger et simple, messages courts et faciles à décoder pour le navigateur et serveur d'application
- Auto-descriptif : navigation intuitive à travers les ressources.
- Stateless: Adapté pour des opérations simples (consulter, supprimer, modifier, etc.)
- Moins de dépendances sur les outils.
- Architecture similaire aux sites internet.

Inconvénients

- Manque de complexité exigés par les transactions d'affaires
- REST est incomplet pour fournir une solution fiable aux entreprises lors d'échanges complexes.



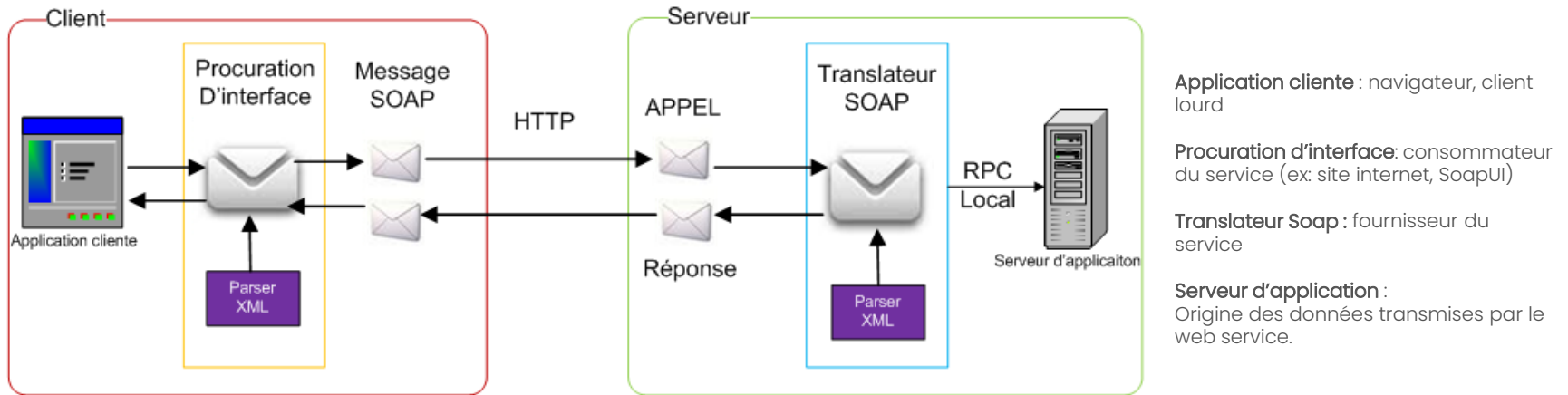
SOAP vs REST



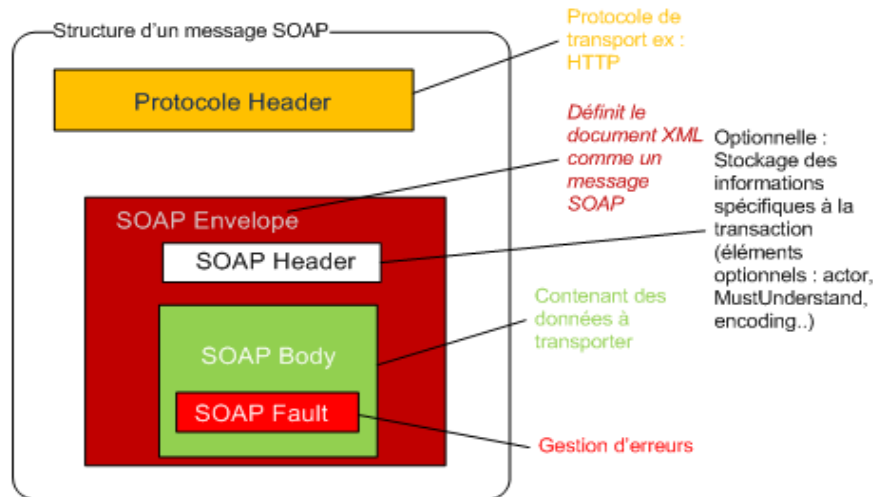
Source : <https://dzone.com/articles/rest-api-vs-soap-web-services-management>

Architecture du protocole SOAP

- L'invocation de méthodes lors d'une communication Soap entre des serveurs peut être schématisée de la manière suivante :



- La communication est réalisée par l'envoi de messages XML. Voici la structure d'un message SOAP.



Exemple de messages SOAP

- CurrencyConverter est un service web Soap libre d'accès. Il propose une opération « ConversionRate » accessible sur 2 protocoles (Soap 1.1 et Soap 1.2).
- Voici une trace des échanges Soap effectués lors de la communication entre le serveur et le client :

→ Requête SOAP (client)

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:web="http://www.webserviceX.NET/">
  <soapenv:Header/>
  <soapenv:Body>
    <web:ConversionRate>
      <web:FromCurrency>EUR</web:FromCurrency>
      <web:ToCurrency>USD</web:ToCurrency>
    </web:ConversionRate>
  </soapenv:Body>
</soapenv:Envelope>
```

→ Réponse SOAP (serveur)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <soap:Body>
    <ConversionRateResponse xmlns="http://www.webserviceX.NET/">
      <ConversionRateResult>1.0849</ConversionRateResult>
    </ConversionRateResponse>
  </soap:Body>
</soap:Envelope>
```

Données transmises

Soap Body (données échangées)

Données renvoyées

Exemple de message REST

Structure du message REST

Requête *GET https://eshop.com/books/14675*



Réponse

HTTP status : 200 OK

*Allow : GET, HEAD, OPTIONS
content-type : application/json
vary : Accept*

```
{  
  "id": "14675",  
  "title": "Game of Thrones",  
  "author": "George R. R. Martin",  
  "isbn": "1-84356-028-3",  
  "availableQty": "4"  
}
```

Commandes et codes HTTP

Les commandes permettent de spécifier le type de requête et la méthode à utiliser. Voici une liste des commandes :

Commande	Description
GET	C'est la méthode la plus courante pour demander une ressource. Une requête GET est sans effet sur la ressource, il doit être possible de répéter la requête sans effet.
HEAD	Cette méthode ne demande que des informations sur la ressource, sans demander la ressource elle-même.
POST	Cette méthode doit être utilisée lorsqu'une requête modifie la ressource.
OPTIONS	Cette méthode permet d'obtenir les options de communication d'une ressource ou du serveur en général.
CONNECT	Cette méthode permet d'utiliser un proxy comme un tunnel de communication.
TRACE	Cette méthode demande au serveur de retourner ce qu'il a reçu, dans le but de tester et d'effectuer un diagnostic sur la connexion.
PUT	Cette méthode permet d'ajouter une ressource sur le serveur.
DELETE	Cette méthode permet de supprimer une ressource du serveur.

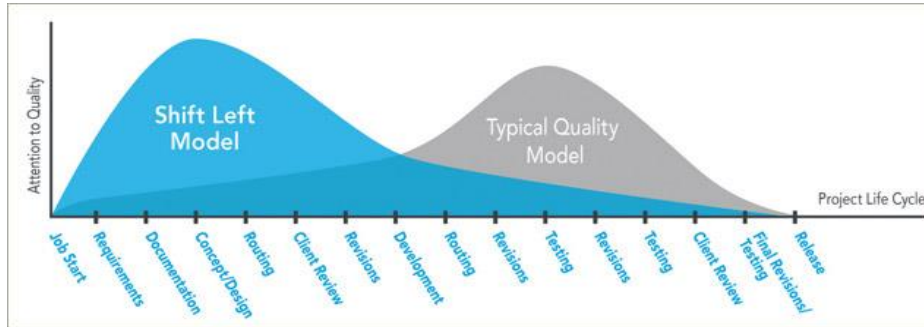
- Les codes permettent de renvoyer le résultat de l'exécution d'une requête par un serveur à un client :

Groupe	Exemple (Code, message et description)
Information	100 : Continue . Attend la suite de la requête
Succès	200 : OK . Requête traitée avec succès 201 : Created . Requête traitée avec succès avec création d'un document
Redirection	301 : Moved Permanently . Document déplacé de façon permanente 302 : Moved Temporarily . Document déplacé de façon temporaire
Erreur du client / du serveur web	401 : Unauthorized . Une authentification est nécessaire pour accéder à la ressource 403 : Forbidden . Le serveur a compris la requête, mais refuse de l'exécuter 404 : Not Found . Ressource non trouvée
Erreur du serveur / du serveur d'application	500 : Internal Server Error . Erreur interne du serveur 502 : Bad Gateway ou Proxy Error . Mauvaise réponse envoyée à un serveur intermédiaire par un autre serveur.

Pourquoi tester les API ?

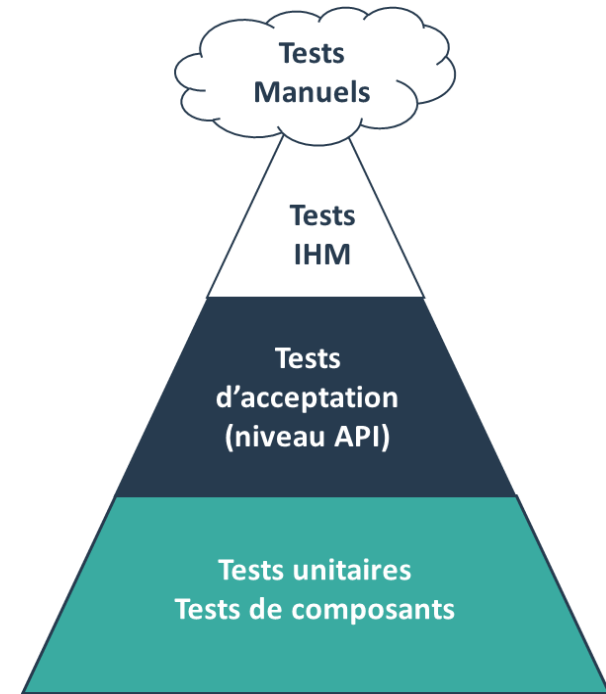
- Les tests IHM présentent des inconvénients
 - Ils ne peuvent être exécutés que lorsque l'IHM est prête
 - Leur temps d'exécution est souvent très long (manuel)
 - Les scripts automatisés sur l'IHM sont souvent très sensibles aux changements et donc difficiles à maintenir
- Exemples :
 - Une suite de tests de régression qui prend plusieurs heures à être exécutée n'est pas viable sur un cycle d'intégration continue où les développeurs ont besoin d'un feedback rapide sur leur build
 - Le développement d'une nouvelle appli où la logique métier est surtout contenue dans les APIs

Pourquoi tester les API ?



Shift Left est une approche utilisée pour accélérer les tests logiciels et faciliter le développement en passant le processus de test à un point antérieur du cycle de développement.

Le décalage vers la gauche est une référence au déplacement des tests vers la gauche sur une ligne de temps.

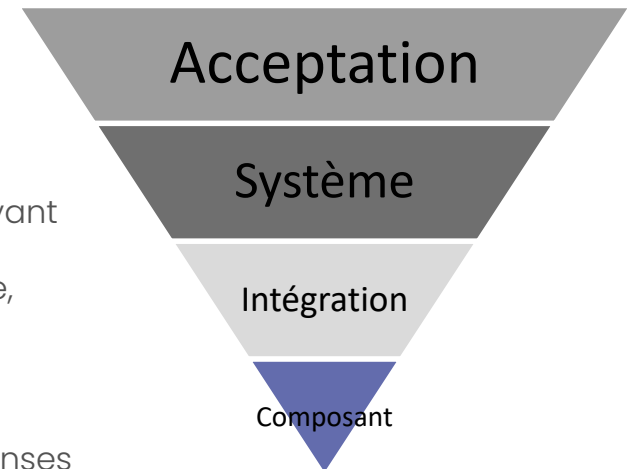


Pyramide de Mike Cohn ©

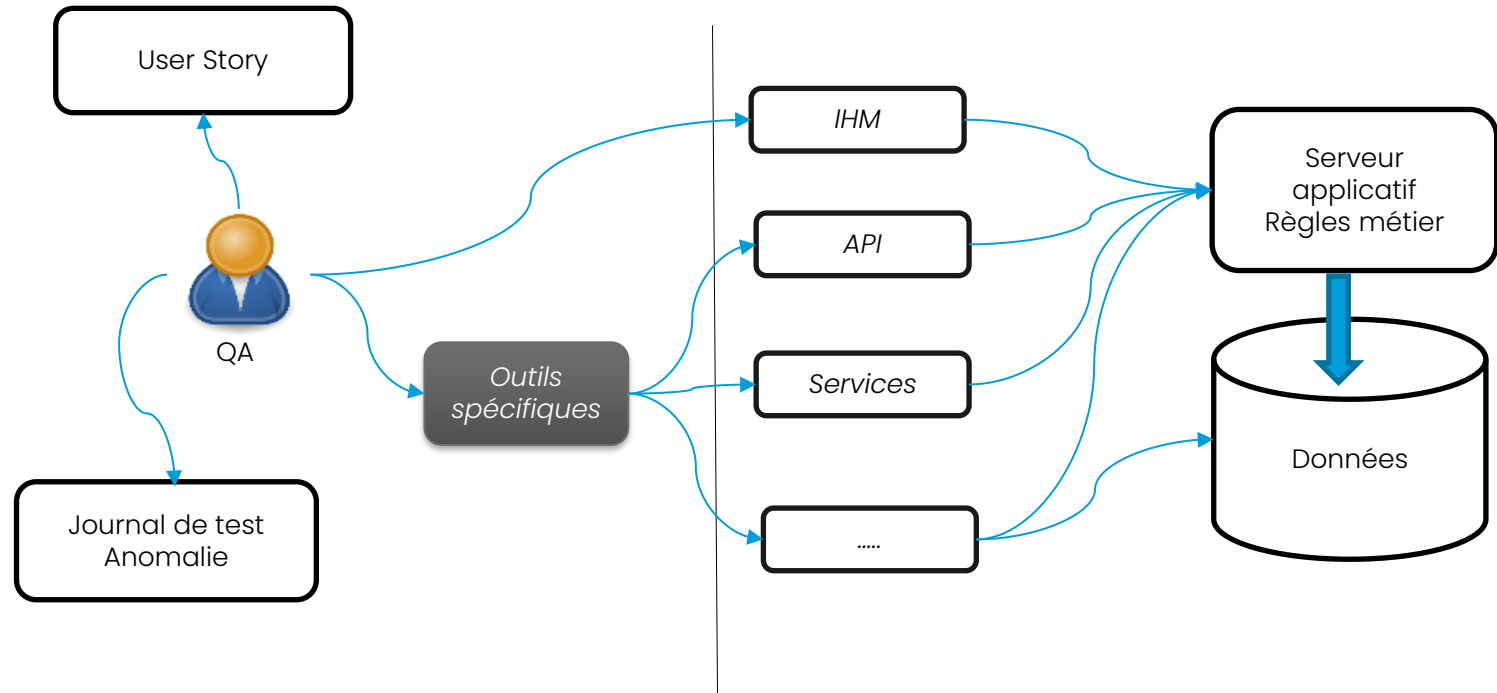
- Les tests automatiques IHM sont difficiles à maintenir.
- La cohérence entre les différentes activités de tests automatiques est nécessaire pour une meilleure couverture des risques.
- Questions :
 - Quel est le niveau de test le plus pertinent pour l'automatisation ? → Gherkin (cucumber.io)
 - Comment rendre les tests automatiques IHM facile à maintenir ? → Framework mots-clefs

Comment tester des API

- Périmètre de test
 - Implémentation cachée de l'application (test sous l'IHM)
- Niveaux de test
 - Composant : API individuelles
 - Intégration : scénarios de bout-en-bout
- Objectifs de test
 - Focaliser sur le fonctionnement attendu des composants avant intégration dans l'application
 - Vérifier les caractéristiques non fonctionnelles (performance, robustesse, sécurité...)
- Techniques de test
 - Utiliser un outil pour envoyer des requêtes et vérifier les réponses
 - Simuler les données ou les flux de contrôle



Rôle du testeur Agile



Assurer la cohérence de la couverture des risques par rapport à l'effort de test

Comment tester des API

- Etape 1 : savoir ce que l'API doit faire
 - Documentation (swagger)
 - Information obtenue des développeurs
 - Capture de traces
- Etape 2 : écrire des scénarios de test avec vérification des résultats attendus
 - Statut code
 - Présence du message retourné
 - Contenu du message retourné
 - Temps de réponse
- Etape 3 : implémenter les tests dans l'outil
- Etape 4 : exécuter les tests avec l'outil

Synthèse des tests API



- Très utiles pour l'équipe de développement et de test
- Très efficaces pour trouver des bugs lors de l'intégration de l'application
- Peuvent vérifier les caractéristiques fonctionnelles et non fonctionnelles de l'application
- Leur automatisation est plus rapide et plus stable que pour les tests IHM
- Apportent une visibilité plus rapide à l'équipe de développement sur le fonctionnement de l'application



- Pas d'interface disponible pour préparer les tests
- Pas de vérification de l'utilisabilité de l'interface (IHM)
- Nécessitent des outils spécifiques
- Demandent des testeurs avec des compétences additionnelles



Savoir parler des API sans être un expert

Par Guillaume ADIN, Jean-Arnaud GAUTHUN et Yacine DECHMI

Définition API

API ou **Application Programming Interface** est une interface de programmation d'application proposant un ensemble de fonctions informatique normalisées qui permettent à une application d'exposer, communiquer et échanger du contenu avec d'autres applications.

Les 2 grands usages des API

Fourniture de Service

Exposer un service de bout en bout :

- Service d'authentification (google, facebook)
- Service de cartographie (google maps)
- Service de paiement (Apple Pay, PayPal, PayLib)
- Service intelligence artificielle (IBM Watson)

Partage de données

Partager des données en interne ou en externe

- Partager des données RH dans le SI
- Echanger des données entre deux applications
- Exposer des données à des partenaires

Les erreurs à éviter

Ne pas définir de format d'échange entre les API ou vers les API : complexité d'accès + maintenance à risque

Ouvrir l'accès à tous : Rendre visible des données confidentielles

Ne pas mettre de file d'attente : Surcharger le gestionnaire d'API

Ne pas déployer de cache* pour gérer les files d'attente de requêtes : Surcharger le gestionnaire d'API

Ne pas communiquer sur les modes d'accès aux API : Impossibilité d'utiliser les API

Ne pas communiquer sur les existences des API dans l'entreprise : Duplication des API

**Toutes les réponses à des requêtes identiques sur un laps de temps (à définir) sont les mêmes*

L'API management

La gestion des API permet de gérer le peuplement des API dans SI, et permet de :

- Gérer le dispatch sur les API (empêcher les plus demandées soient saturées) – Passerelle API
- Mettre les API à disposition de façon sécurisée
- Gérer le cycle de vie des APIs
- Fournir de l'information sur l'utilisation de l'API
- Monétiser l'utilisation des API

Matrice d'évaluation by onepoint

Gouvernance

- Stratégie
- Gouvernance
- Considération juridique

API Engineering

- Design
- Implémentation
- Qualité
- Infrastructure

Business Model

- Chaine de valeur
- Marketing
- Documentation

Cyber Sécurité

- Règles de sécurité
- Traçabilité
- Audibilité
- Sécurisation de l'API

Support

- Opérer les APIs
- Support offert aux développeurs

Un exemple concret





2

Quiz

1.Quel est l'objectif principal des tests API ?

- A) Vérifier l'interface utilisateur
- B) Trouver des bugs lors de l'intégration de l'application
- C) Tester uniquement les performances du système

2.Quels sont les niveaux de test définis par le syllabus de l'ISTQB ?

- A) Tests unitaires, tests d'intégration, tests de performance
- B) Tests composants, tests d'intégration, tests système, tests d'acceptation
- C) Tests fonctionnels, tests de régression, tests de charge

3.Quelle technique de test est souvent utilisée pour les tests API ?

- A) Test en boîte noire
- B) Test en boîte grise
- C) Test en boîte blanche

4.Quel est un des avantages des tests API par rapport aux tests d'interface utilisateur (IHM) ?

- A) Ils sont plus lents à automatiser
- B) Ils nécessitent moins de compétences techniques
- C) Leur automatisation est plus rapide et plus stable

5.Pourquoi les applications doivent-elles être flexibles selon le document ?

- A) Pour réduire les coûts de développement
- B) Pour s'adapter à différents environnements et intégrer des changements rapidement
- C) Pour éviter les tests de régression

1. Quel type de tests est principalement concerné par l'analyse des valeurs limites ?

- A) Tests de performance
- B) Tests de sécurité
- C) Tests en boîte noire

2. Quel est un inconvénient des tests API mentionné dans le document ?

- A) Ils sont trop coûteux
- B) Ils nécessitent des outils spécifiques
- C) Ils ne peuvent pas être automatisés

3. Quel est le rôle des interfaces entre les composants d'une application ?

- A) Elles ralentissent le système
- B) Elles sont essentielles au bon fonctionnement du système complet
- C) Elles ne sont pas nécessaires si les composants sont bien conçus

4. Quel type de test est le plus adapté pour vérifier les caractéristiques non fonctionnelles d'une application ?

- A) Tests d'intégration
- B) Tests API
- C) Tests d'acceptation

5. Quel est un des conseils donnés pour différencier les niveaux de test ?

- A) Par le temps nécessaire pour les réaliser
- B) Par les vérifications effectuées et les risques couverts
- C) Par le nombre de testeurs impliqués



3

Installation et configuration Postman

Pourquoi utiliser Postman ?

- *Un outil gratuit* de test des webservices principalement en REST.
- *Accessibilité et convivialité*
- Peut-être implémenté sur *multiple système d'exploitation* : Windows, Mac OS, Linux.
- *Une large communauté*
- Les appels d'API *sans avoir à coder des scripts*.
- Fonctionnalités riches.
- Largement répondu.



Pré-requis

1. Poste Windows 10/11
2. Compte Postman actif : <https://identity.getpostman.com/signup>
3. Télécharger l'application et l'installer : <https://www.postman.com/downloads/>
4. Télécharger l'application FlightApi.zip et dézipper le contenu dans un dossier de travail
https://github.com/imhah-hassan/Postman_training/blob/master/Demo%20App/FlightApi.zip

Vue d'ensemble

POSTMAN - API DEVELOPMENT PLATFORM



Installation

Télécharger Postman : <https://www.postman.com/downloads/>

The Postman app

Download the app to get started with the Postman API Platform.

 Windows 64-bit

 Postman-win64-Se....exe ^



Connectez vous avec le compte que vous venez de créer !

Vous pouvez créer un nouveau compte

Create an account or sign in

Create Free Account

Sign in

Sign In

[Create Account](#) instead?

Email or Username

p.le-huu@groupeonepoint.com

Password

.....

☒ Stay signed in for 30 days

[Forgot Password?](#)

Sign in

onepoint.

La page d'accueil

Diagram illustrating the layout of the Postman homepage:

- Page d'accueil** (Home)
- Où nos travaux seront organisés** (Where our work will be organized)

The interface shows the following elements:

- Navigation bar: Home, Workspaces, API Network, Explore, Search Postman, Upgrade, and utility icons.
- Greeting: **Good afternoon, bruno!** with the message "Pick up where you left off."
- Left sidebar: "Postman works best with teams" section with a "Create Team" button and a "Workspaces" link.
- Right panel: "Take a shortcut to sending requests" section with a "Send a request" button and a request editor.

Request Editor Details:

- Method: GET
- URL: `https://postman-echo.com/get`
- Buttons: Send, Cookies
- Tabs: Params, Auth, Headers (7), Body, Pre-req, Tests, Settings
- Section: Query Params
- Table:

KEY	VALUE	DESCRIPTION	Bulk Edit
key	value	Description	

Création du Workspace

Créer un espace de travail pour chacun de vos projets

The image shows two parts of the Postman interface. On the left, the 'Workspaces' dropdown menu is open, showing a search bar, a 'Create Workspace' button (marked with a red circle 1), and a list of recently visited workspaces. On the right, the 'Create workspace' form is shown, with fields for 'Name' (marked with a red circle 2), 'Summary' (marked with a red circle 3), and 'Visibility' (marked with a red circle 3). The 'Visibility' section has radio buttons for 'Personal', 'Private', 'Team', 'Partner', and 'Public'. The 'Create Workspace' button is marked with a red circle 4.

Création du nouveau Workplace

1

2

3

4

Retrouver ici la liste de vos espaces de travail

Saisir le nom du projet

Choisir Personal

Valider

Création d'une collection

Créer une collection (ensemble de requêtes présentant avec une cohérence)

Création d'une nouvelle collection

Saisir le nom de la collection

Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

New Collection
This collection is empty
[Add a request](#) to start working.

Exercice - Requetes Simple

Auth Pre-req. Tests Variables Runs

This authorization method will be used for every request in this collection. You can override this by specifying one in the request.

Type No Auth

Ou clic droit sur la collection, et choisir Rename

Structure

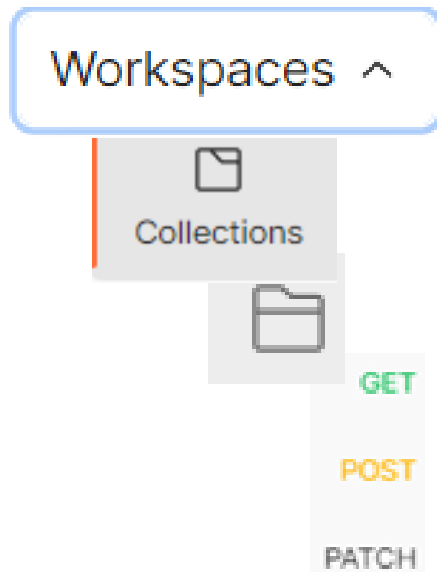
Organisation des requêtes :

- Chaque Workspace peut contenir plusieurs collections
- Chaque collection peut être organisée avec des dossiers
- Les requêtes peuvent être ajoutées dans un dossier ou directement dans la collection

Organisation des variables

- Les variables peuvent être globales ou liées à une collection.
- Les variables peuvent aussi être organisées dans les environnements pour simplifier la maintenance.

Requêtes



Variables

Globals



Exercice 0 : Préparation

- 4. Préparation
 - 4.1. Création d'un workspace
 - 4.2. Création de collection
 - 4.3. Importer / Exporter une collection
 - 4.4. Création d'une requête API
 - 4.5. Liste des requêtes FlightsAPI utilisées

Documentation de l'API Flights :

<https://app.swaggerhub.com/apis/imhah-hassan/FligthApi/Mars2023>

3

Création des requêtes simples

<https://app.swaggerhub.com/apis/imhah-hassan/PyFlightsApi/Dec22#/>

- Consultation (GET)
- Création (POST)
- Mise à jour (PATCH)
- Suppression (DELETE)

Requête Rest dans Postman

GET  `https://api.taiga.io/api/v1/issues?project=404046`

Params • Authorization Headers (8) Body Pre-request Script Tests Settings

Headers  7 hidden

	KEY	VALUE
☒	Content-Type	application/json

Params • Authorization Headers (8) Body Pre-request Script

Query Params

	KEY	VALUE
☒	project	404046

Params Authorization Headers (10) **Body •**

☐ none ☐ form-data ☐ x-www-form-urlencoded 

```
1 {
2   "password": "Hassan$2022",
3   "type": "normal",
4   "username": "h.imhah"
5 }
```

Params **Authorization** Headers (10)

Type  Inherit a...

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#) >

Params Authorization Headers (10) Body • **Pre-request Script** Tests •

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Body matches auth_token string", function () {
6   pm.expect(pm.response.text()).to.include("auth_token");
7 });
8
9 pm.test("Verify user name", function () {
10  var jsonData = pm.response.json();
11  pm.expect(jsonData.username).to.eql("h.imhah");
12 });
```

GET  ht

GET

POST

PUT

PATCH

DELETE

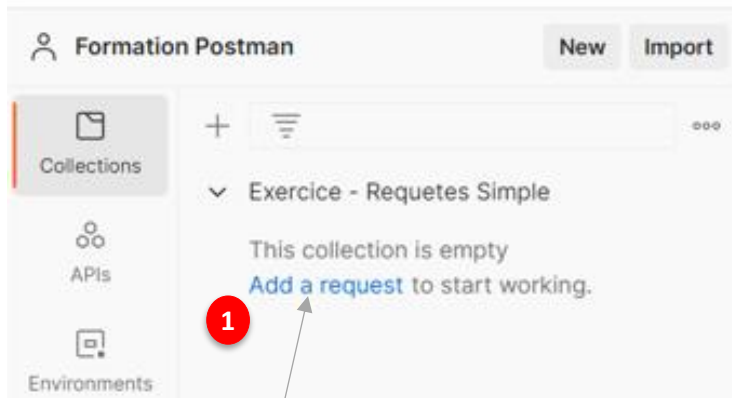
COPY

Params Authorization Headers (11) Body • **Pre-request Script •**

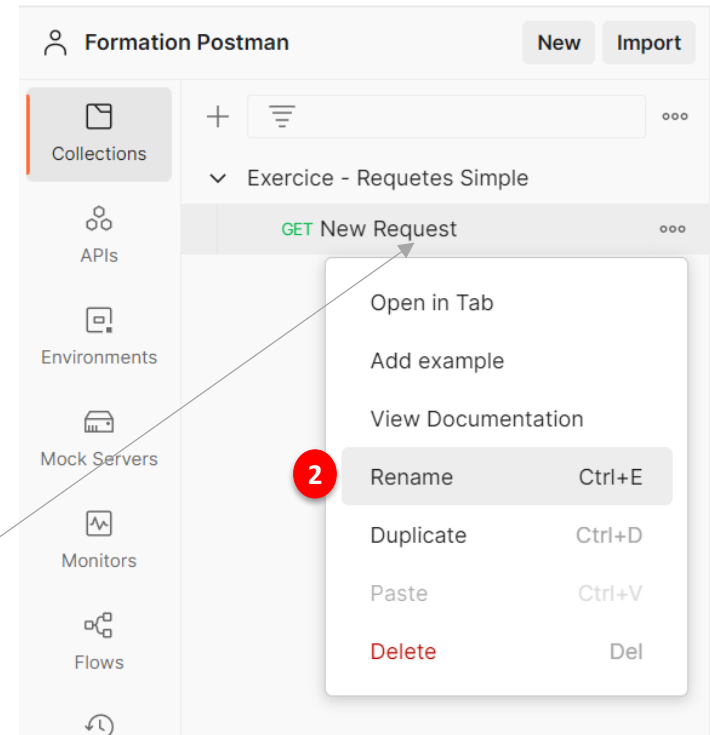
```
1 var issuesCount = pm.environment.get ("issuesCount")
2 console.log (pm.environment.get ("issuesCount"));
3
4 if (issuesCount > 1) {
5   postman.setNextRequest ("issues");
6 }
```

Création d'une requête API

Lancer une requête pour lire les informations d'un vol via son numéro.

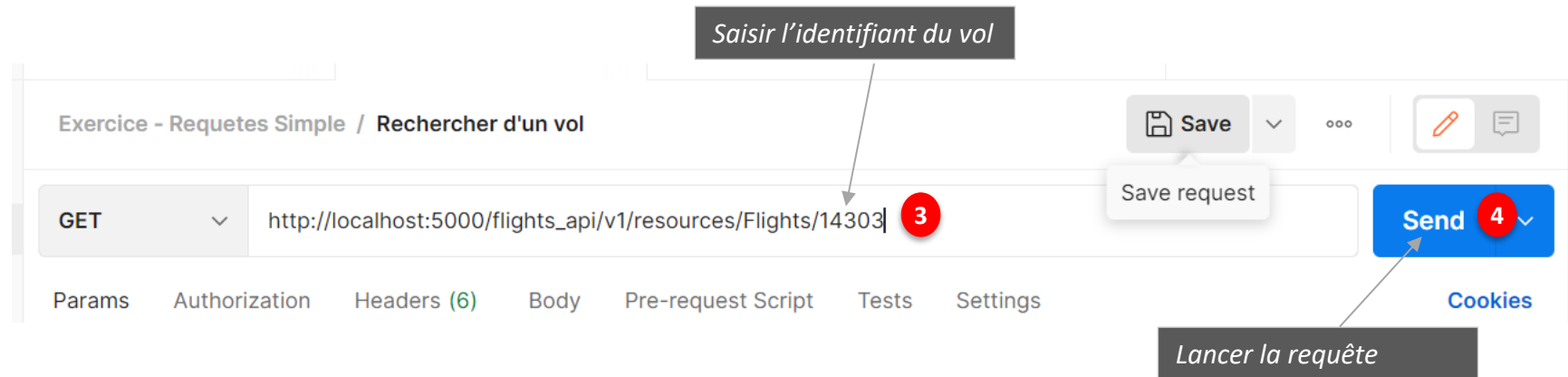
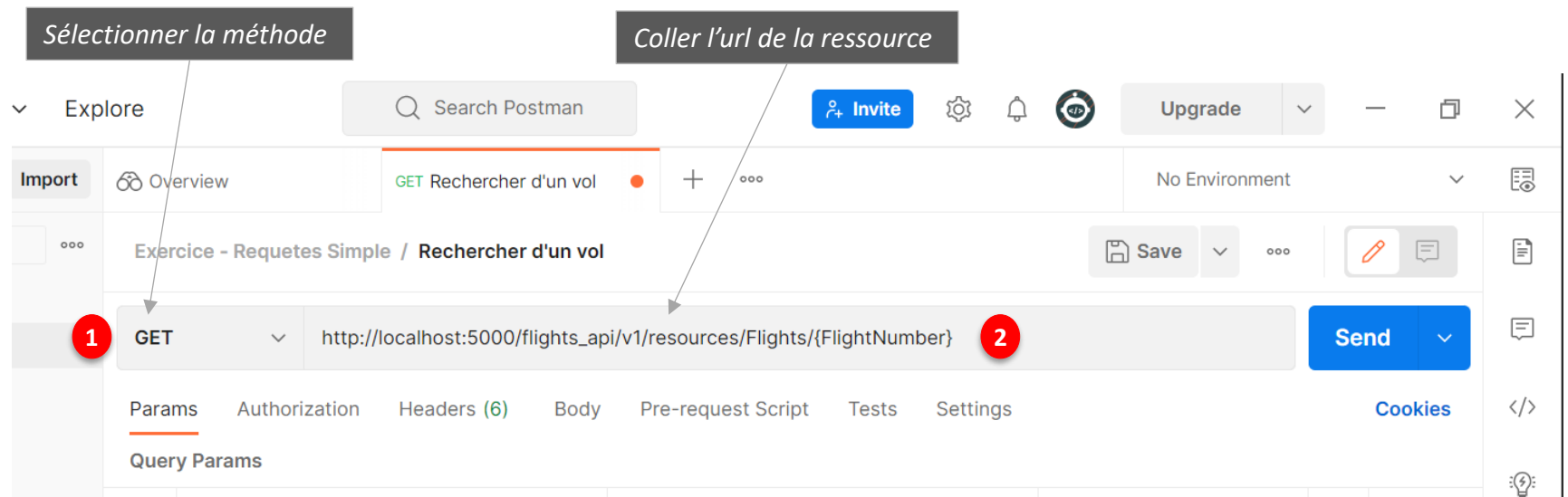


Création d'une requête (Add a request)



Clic droit sur la requête, puis choisir Rename

Création d'une requête simple (GET)



N'oublier pas de sauvegarder (**Save**)(ou **CRL + S**) avant de lancer votre requête (**Send**)

Résultat d'une requête simple (GET)

En retour une réponse avec un code 200 (ok) et les informations en format JSON du vol.

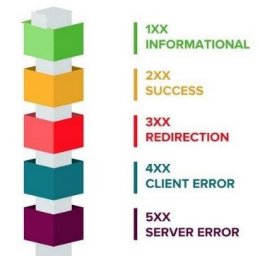
The screenshot shows the Postman interface with a GET request to `http://localhost:5000/flights_api/v1/resources/Flights/14303`. The response is a 200 OK status with a response time of 81 ms and a body size of 360 B. The response body is displayed in JSON format, showing flight details.

KEY	VALUE	DESCRIPTION
Key	Value	Description

```
1 {
2   "Airline": "NW",
3   "ArrivalCity": "Frankfurt",
4   "ArrivalTime": "07:41 PM",
5   "DepartureCity": "Denver",
6   "DepartureTime": "06:57 PM",
7   "FlightNumber": 14303,
8   "Price": 100.8,
9   "SeatsAvailable": 250,
10  "DayOfWeek": "Friday"
11 }
```

Réponse au format json

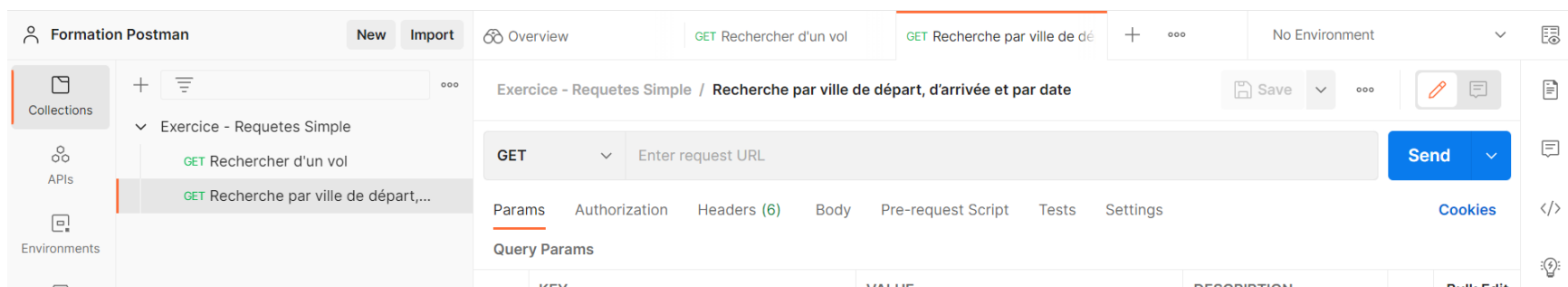
HTTP Status Codes



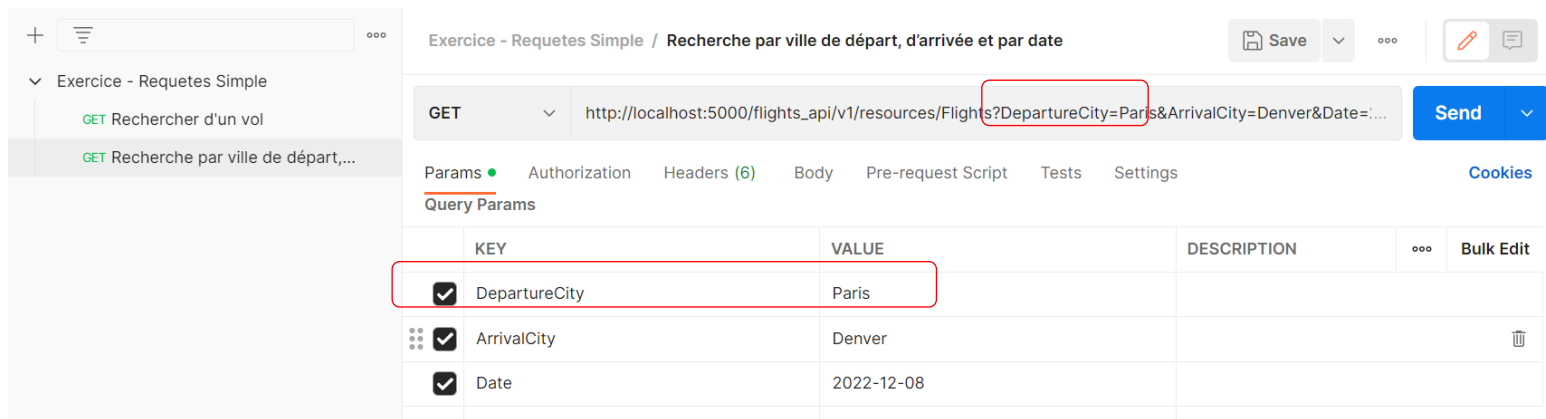
Création d'une requête (GET)

Rechercher un vol par ville de départ, ville d'arrivée et par date

Faire un clic droit sur la collection (Exercice – Requetes Simple), puis choisir le menu **Add Request**
Renommer la requête



Sélectionner la méthode (GET) et coller l'url de la ressource

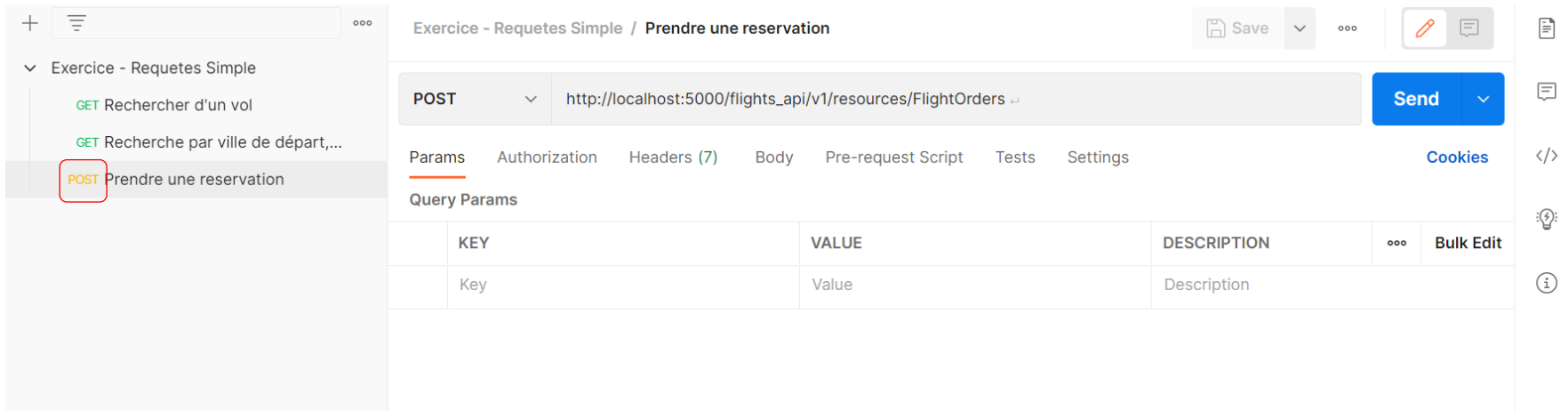


Pour modifier une requête vous pouvez utiliser l'onglet **Params** au lieu de modifier URL,
Par exemple, remplacer la valeur Paris par la valeur London pour le paramètre **DepartureCity**

Création d'une requête (POST)

Prendre une réservation

Créer une requête et la nommer. Sélectionner la méthode (POST) et coller l'url de la ressource



The screenshot shows a REST client interface with a sidebar on the left containing a list of requests. The main panel displays the details of a new request named "Prendre une reservation".

Left Sidebar:

- Exercise - Requetes Simple
 - GET Rechercher d'un vol
 - GET Recherche par ville de départ,...
 - POST Prendre une reservation** (highlighted with a red box)

Main Panel:

Exercise - Requetes Simple / Prendre une reservation

POST http://localhost:5000/flights_api/v1/resources/FlightOrders

Buttons: Save, Send

Tabs: Params, Authorization, Headers (7), Body, Pre-request Script, Tests, Settings, Cookies

Query Params Table:

KEY	VALUE	DESCRIPTION
Key	Value	Description

Buttons: Bulk Edit, Info

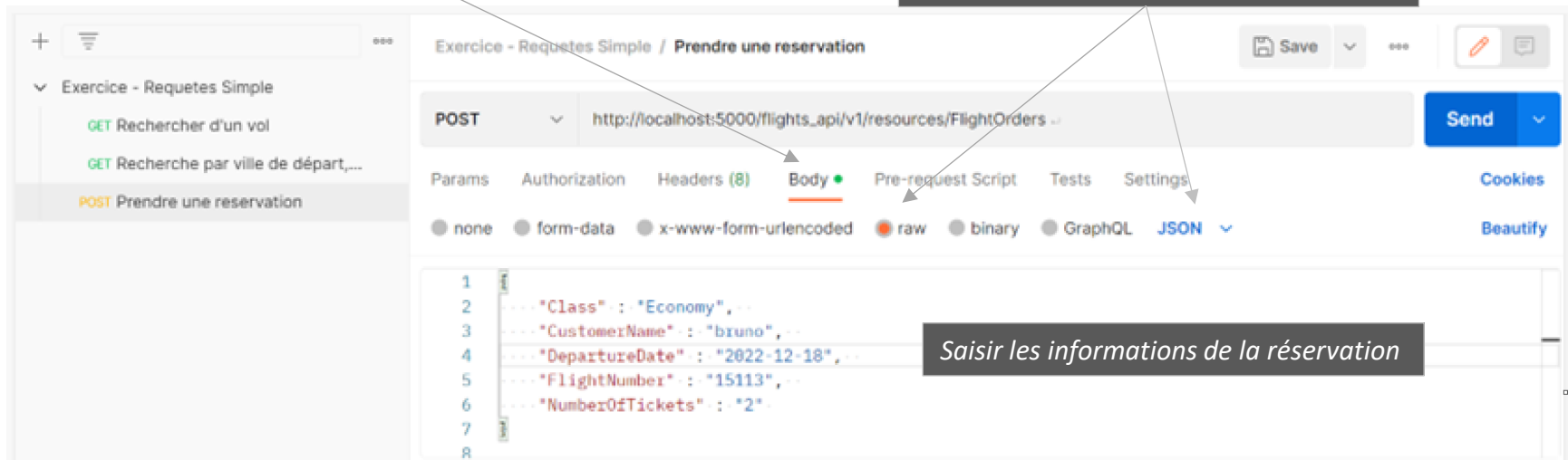
Après la sauvegarde, la requête est tagguée comme étant un requête de type POST

Création d'une requête (POST)

Envoyer les informations de la nouvelle réservation

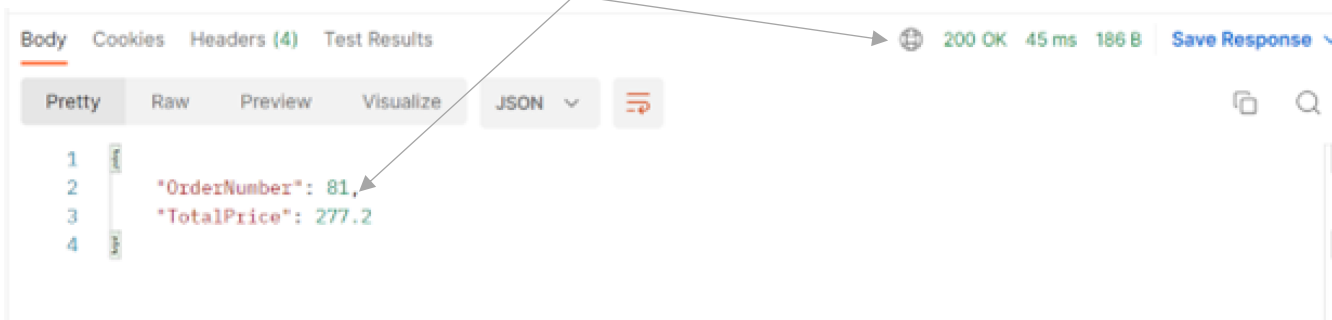
Cliquer sur l'onglet **Body** pour accéder aux formats disponibles

Choisir le format **Raw** et préciser **JSON**



Saisir les informations de la réservation

Enregistrer le requête et envoyer la : Réponse est retourné avec l'identifiant de la réservation



Création d'une requête (PATCH)

Modifier une réservation existante

The screenshot shows a REST client interface with a sidebar on the left and a main workspace. The sidebar lists several requests under 'Exercice - Requetes Simple': 'GET Rechercher d'un vol', 'GET Recherche par ville de départ...', 'POST Prendre une reservation', and 'PATCH Modifier une reservation'. The 'PATCH Modifier une reservation' item is selected. The main workspace shows a PATCH request to 'http://localhost:5000/flights_api/v1/resources/FlightOrders/81'. The request body is a JSON object:

```
{  "Class": "Economy",  "CustomerName": "bruno",  "DepartureDate": "2022-12-18",  "FlightNumber": "15113",  "NumberOfTickets": "5"}
```

. The response is a 200 OK status with a 40 ms response time and 314 B of data. The response body is a JSON object:

```
{  "Class": "Economy",  "CustomerName": "bruno",  "DepartureDate": "2022-12-18 18:15",  "FlightNumber": 15113,  "NumberOfTickets": 5,  "OrderNumber": 81,  "TotalPrice": 222.0}
```

. Annotations with arrows point to specific elements: 'Appeler la réservation à modifier (81)' points to the URL; 'Sélectionner la méthode' points to the PATCH method; 'Saisir les informations de la réservation dont le nouveau nombre de tickets' points to the request body; 'Nouvelle valeur prise en compte' points to the 'NumberOfTickets' field in the response body.

Appeler la réservation à modifier (81)

Sélectionner la méthode

Saisir les informations de la réservation dont le nouveau nombre de tickets

Nouvelle valeur prise en compte

Création d'une requête (DELETE)

Supprimer une réservation existante

The screenshot shows a REST client interface with a sidebar on the left and a main panel on the right. The sidebar lists several API endpoints under the heading "Exercice - Requetes Simple". The main panel displays a DELETE request to the URL `http://localhost:5000/flights_api/v1/resources/FlightOrders/81`. The response is a 200 OK status with a body containing the JSON `"true": "Order deleted 81 "`.

Annotations:

- Appeler la réservation à modifier (81)**: Points to the URL in the request bar.
- Sélectionner la méthode**: Points to the DELETE method in the sidebar.
- Confirmation de la suppression**: Points to the response status and body.

Interface Details:

- Left Sidebar:** Contains a list of API endpoints: GET Rechercher d'un vol, GET Recherche par ville de départ,..., POST Prendre une reservation, PATCH Modifier une reservation, and DEL Supprimer une reservation (highlighted).
- Main Panel:** Shows the DELETE request details, including the URL, method, and response status (200 OK) and body (`"true": "Order deleted 81 "`).

Consulter la log d'exécution

Consultation des logs d'exécution en cliquant sur Console

The screenshot displays a REST client interface. On the left, a sidebar shows 'Collections' with a folder named 'Exercice - Requetes Simple' containing several requests. The 'Console' tab is selected and highlighted with a red box. The main area shows a 'DELETE' request to 'http://localhost:5000/flights_api/v1/resources/FlightOrders/81'. The response is a JSON object: `{ "true": "Order deleted 81" }`. Below the main area, a list of logs is visible, with the 'Console' tab selected. A 'Clear' button is located at the top right of the logs section.

Exercice - Requetes Simple / Supprimer une reservation

DELETE `http://localhost:5000/flights_api/v1/resources/FlightOrders/81` Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Body Cookies Headers (4) Test Results

200 OK 60 ms 174 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "true": "Order deleted 81" }
```

Online Find and Replace Console

- ▶ GET `http://localhost:5000/flights_api/v1/resources/Flights/14303` 200 81 ms
- ▶ POST `http://localhost:5000/flights_api/v1/resources/Flights/14303` 405 9 ms
- ▶ GET `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=2021-12-08` 200 90 ms
- ▶ GET `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=2022-12-08` 200 20 ms
- ▶ GET `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=London&ArrivalCity=Denver&Date=2022-12-08` 200 12 ms
- ▶ POST `http://localhost:5000/flights_api/v1/resources/FlightOrders%0A` 500 127 ms
- ▶ POST `http://localhost:5000/flights_api/v1/resources/FlightOrders%0A` 200 45 ms
- ▶ PATCH `http://localhost:5000/flights_api/v1/resources/FlightOrders/81%0A` 200 41 ms
- ▶ PATCH `http://localhost:5000/flights_api/v1/resources/FlightOrders/81%0A` 200 44 ms
- ▶ PATCH `http://localhost:5000/flights_api/v1/resources/FlightOrders/81%0A` 200 40 ms
- ▶ DELETE `http://localhost:5000/flights_api/v1/resources/FlightsOrders/81` 404 8 ms

Clear

Cookies Capture requests Runner Trash

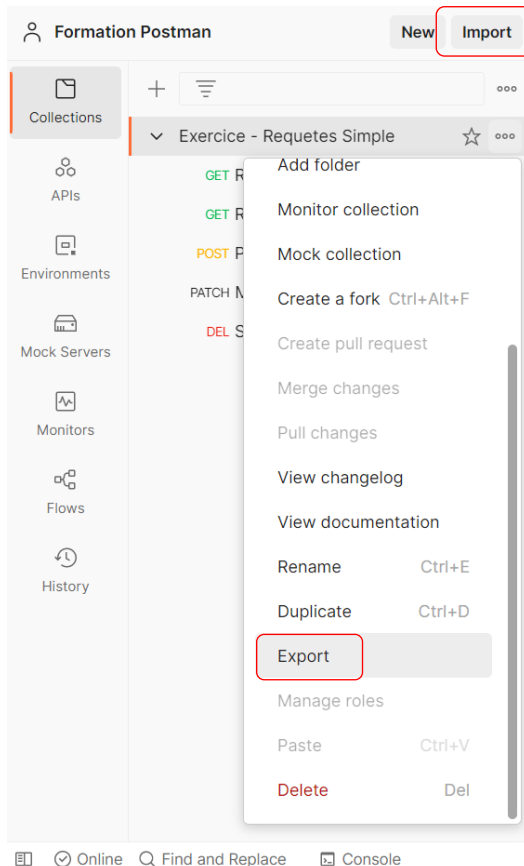
Pour afficher le détail, cliquer sur la requête

Pour effacer le contenu cliquer sur Clear.

Importer / Exporter une collection

Pour exporter une collection, faire un clic droit sur la collection qu'on souhaite exporter,
Cliquer sur le bouton **Export** dans la popup qui s'affiche,
Choisir l'emplacement du fichier JSON et confirmer l'enregistrement..

Pour l'import d'une collection, cliquer sur le bouton « Import », puis naviguer jusqu'à votre fichier de collection



Exercice 1 : Rechercher un vol

Utiliser le tutorial

- 5. Exercice 1 : Rechercher un vol
 - 5.1. Requête GetFlight : recherche par numéro
 - 5.2. Requête GetFlights : recherche par ville de départ, d'arrivée et par date
 - 5.3. Consulter le log d'exécution



4

Ajout des vérifications

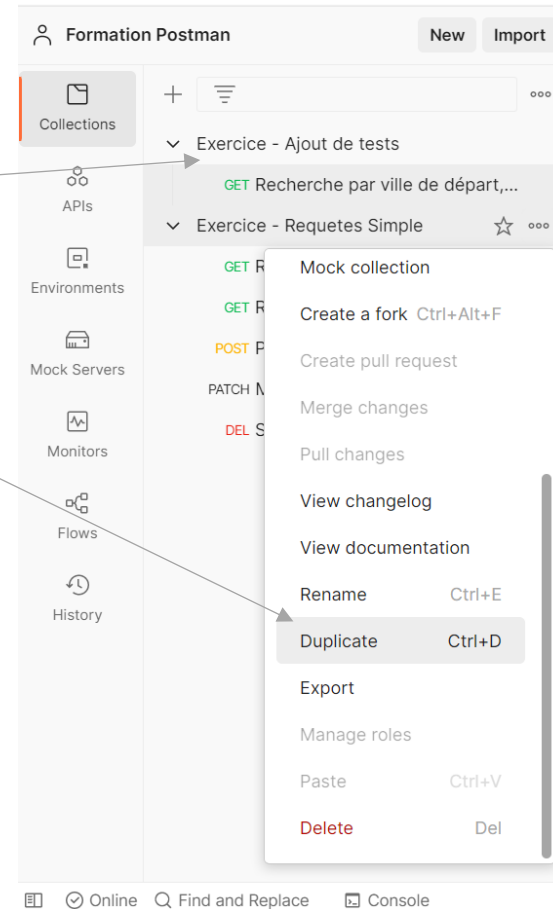
<https://learning.postman.com/docs/writing-scripts/script-references/test-examples/>

Vérifier un code retour

Préparation d'une nouvelle collection

Création d'une nouvelle collection

*Dupliquer la requête :
Recherche par ville de départ, d'arrivée et par date,
Puis déplacer la vers la nouvelle collection*



Note : On peut également dupliquer une collection

Vérifier un code retour

Mise au point d'un premier test

Cliquer sur l'onglet Tests

*Cliquer sur le Snippets **Status code: Code is 200***

1

2

3

Le code JavaScript est ajouté automatiquement

Exercice - Ajout de tests / Recherche par ville de départ, d'arrivée et par date Copy

GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...

Params Authorization Headers (6) Body Pre-request Script **Tests** Settings Cookies

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
```

Test scripts are written in JavaScript, and are run after the response is received. Learn more about [tests scripts](#)

Snippets

[Send a request](#)

[Status code: Code is 200](#)

[Response body: Contains string](#)

[Response body: JSON value check](#)

Body Cookies Headers (4) Test Results

200 OK 24 ms 1.4 KB Save Response

Pretty Raw Preview Visualize

```
[{"Airline": "AF", "ArrivalCity": "Denver", "ArrivalTime": "10:30 AM", "DepartureCity": "Paris", "DepartureTime": "08:00 AM", "FlightNumber": 17105, "Price": 139.47, "SeatsAvailable": 250, "DayOfWeek": "Thursday"}, {"Airline": "AF", "ArrivalCity": "Denver", "ArrivalTime": "12:54 PM", "DepartureCity": "Paris", "DepartureTime": "10:24 AM", "FlightNumber": 17109, "Price": 143.47, "SeatsAvailable": 250, "DayOfWeek": "Thursday"}, {"Airline": "AF", "ArrivalCity": "Denver", "ArrivalTime": "01:18 PM", "DepartureCity": "Paris", "DepartureTime": "11:00 AM", "FlightNumber": 17113, "Price": 143.47, "SeatsAvailable": 250, "DayOfWeek": "Thursday"}]
```

Find and Replace Console Cookies Capture requests Runner Trash

Notes : N'oublier pas de sauvegarder

Un snippet (« **extrait** ») est une partie d'un code source pouvant être copié-collé dans un modèle plus large.

Vérifier un code retour

Exécutions de la requête suivi de la vérification

1 - Cas passant :

La vérification est au statut **Pass**

The screenshot shows the Postman interface for a test titled "Recherche par ville de départ, d'arrivée et par date Copy". The test is a GET request to `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...`. The test script is as follows:

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
```

The test results show a status of "200 OK" with a response time of 14 ms and a body size of 1.4 KB. The test is marked as "PASS".

2 - En modifiant le nom de la ville par une ville non référencée, le test devient non passant :

La vérification est au statut **Fail**

The screenshot shows the Postman interface for the same test, but with the URL modified to `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Parisparis&ArrivalCity=Denver&D...`. The test script remains the same. The test results show a status of "500 INTERNAL SERVER ERROR" with a response time of 10 ms and a body size of 449 B. The test is marked as "FAIL".

KEY	VALUE	DESCRIPTION
☒ DepartureCity	Parisparis	
☒ ArrivalCity	Denver	
☒ Date	2022-12-08	

Vérifier le contenu d'une balise Json

Objectif :

Vérifier que la première balise FlightNumber a pour valeur 17105 est présent

Ajouter les Snippets **Response body : Contains String** et **Response body :Json value check**

Exercice - Ajout de tests / Recherche par ville de départ, d'arrivée et par date Copy

GET `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...` Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("Le premier vol est le 17105", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData[0].FlightNumber).to.eql(17105);
12 };
13
```

Test scripts are written in JavaScript, and are run after the response is received. Learn more about [tests scripts](#)

Snippets

- Status code: Code is 200
- Response body: Contains string
- Response body: JSON value check
- Response body: Is equal to a string
- Response headers: Content-Type header

Body Cookies Headers (4) Test Results (3/3) 200 OK 11 ms 1.4 KB Save Response

Pretty Raw Preview Visualize

1^{er} vol {
2^{ème} vol {

```
[{"Airline": "AF", "ArrivalCity": "Denver", "ArrivalTime": "10:30 AM", "DepartureCity": "Paris",  
  "DepartureTime": "08:00 AM", "FlightNumber": 17105, "Price": 139.47, "SeatsAvailable": 250, "DayOfWeek":  
  "Thursday"}, {"Airline": "AF", "ArrivalCity": "Denver", "ArrivalTime": "12:54 PM", "DepartureCity": "Paris",  
  "DepartureTime": "10:24 AM", "FlightNumber": 17109, "Price": 143.47, "SeatsAvailable": 250, "DayOfWeek":
```

Vérifier le contenu d'une balise Json

Résultat :

The screenshot displays a REST client interface with a sidebar on the left containing a list of exercises. The main panel shows a GET request to `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...`. The 'Tests' tab is active, showing three test scripts in JavaScript. The 'Test Results' tab at the bottom shows three passed tests.

Exercise - Ajout de tests / Recherche par ville de départ, d'arrivée et par date Copy

GET `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...` **Send**

Params • Authorization Headers (6) Body Pre-request Script **Tests •** Settings Cookies

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("Le premier vol est le 17105", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData[0].FlightNumber).to.eql(17105);
12 }
```

Test scripts are written in JavaScript, and are run after the response is received. Learn more about [tests scripts](#)

Snippets

- Status code: Code is 200
- Response body: Contains string
- Response body: JSON value check
- Response body: Is equal to a string

Body Cookies Headers (4) **Test Results (3/3)** 200 OK 41 ms 1.4 KB **Save Response**

All Passed Skipped Failed

- PASS** Le code retour est 200
- PASS** La réponse contient le tag FlightNumber
- PASS** Le premier vol est le 17105

Synthèse des tests

Test	Description
<code>pm.response.to.have.status(200);</code>	Le statut http est 200
<code>pm.expect(pm.response.code).to.be.oneOf([201,202]);</code>	Le statut http est parmi ces valeurs
<code>pm.expect(pm.response.responseTime).to.be.below(100);</code>	Le temps de réponse en ms
<code>pm.response.to.have.header('X-Cache');</code>	Le header existe
<code>pm.expect(pm.response.headers.get('X-Cache')).to.eql('HIT');</code>	Le header a cette valeur
<code>pm.expect(pm.cookies.has('sessionId')).to.be.true;</code>	Le cookie existe
<code>pm.expect(pm.cookies.get('sessionId')).to.eql('ad3se3ss8sg7sg3');</code>	Le cookie a cette valeur
<code>pm.response.to.have.body("OK");</code> <code>pm.response.to.have.body({'success'=true});</code>	Le corps de la réponse contient une valeur
<code>pm.expect(pm.response.text()).to.include('Order placed.');</code>	Le texte de la réponse contient une valeur
<code>const response = pm.response.json();</code> <code>pm.expect(response.age).to.eql(30);</code>	Parser la réponse au format json et vérifier une valeur.

Exercice 2 : Les tests

Utiliser le tutorial

- 6. Exercice 2 : Les tests
 - 6.1. Valider le code retour de la requête
 - 6.2. Vérifier la présence d'un texte dans la réponse
 - 6.3. Vérifier le contenu d'une balise JSON



5

Enchainement de plusieurs requêtes

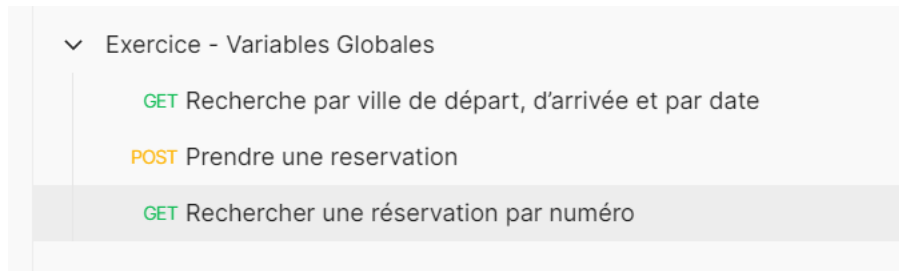
<https://learning.postman.com/docs/writing-scripts/script-references/variables-list/>

Préparation d'une collection dédiée

Objectif: **Simuler le scénario d'utilisation suivant**

- Rechercher les vols disponibles avec une ville de départ, une ville d'arrivée et une date
Sélectionner le 1^{er} vol disponible
- Prendre une réservation sur ce vol
- Contrôler l'existence de la réservation

Création d'une nouvelle collection en dupliquant les requêtes suivantes



Note : la dernière requête n'a pas encore été vu dans cette présentation
GET `http://localhost:5000/flights_api/v1/resources/FlightOrders/{order}`

Enregistrer dans une variable globale

1^{er} requête : Reprendre la requête *Recherche par ville de départ, d'arrivée et par date*

Ajouter le snippet *Set a global variable*,

puis affecter la variable `Numéro_de_vol` avec la valeur du numéro de vol (`jsonData[0].FlightNumber`)

The screenshot displays the Postman interface for a REST client. The left sidebar shows a collection of requests under 'Exercice - Variables Globales'. The main panel shows a GET request to `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...`. The 'Tests' tab is active, showing a JavaScript script that checks for the presence of 'FlightNumber' in the response and sets a global variable 'Numero_du_Vol' to the first flight number found. The 'Test Results' tab shows three passed tests: 'Le code retour est 200', 'La réponse contient le tag FlightNumber', and 'Le premier vol est le 17105 et la variable globale 'Numero_de_vol' est créée'. The 'Snippets' panel on the right shows the 'Set a global variable' snippet selected.

```
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("Le premier vol est le 17105 et la variable globale
  'Numero_de_vol' est créée", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData[0].FlightNumber).to.eql(17105);
12
13   pm.globals.set("Numero_du_Vol", jsonData[0].FlightNumber);
14 }
15 );
```

Test Results (3/3)

- PASS Le code retour est 200
- PASS La réponse contient le tag FlightNumber
- PASS Le premier vol est le 17105 et la variable globale 'Numero_de_vol' est créée

Note : Attention, déposer ce snippet dans *Response body :Json value check*, càd juste avant son accolade fermante

Utiliser la variable enregistrée

2^{ème} requête : Reprendre la requête *Prendre une reservation* et ajouter la variable globale

Sur l'onglet **Body**, positionner la variable globale `"{{Numero_de_vol}}"` pour le champ **FlightNumber**

The screenshot shows a REST client interface with the following details:

- Exercise - Variables Globales / Prendre une reservation**
- POST** `http://localhost:5000/flights_api/v1/resources/FlightOrders`
- Body** tab selected, showing a JSON payload:

```
1 {
2   "Class": "Economy",
3   "CustomerName": "bruno",
4   "DepartureDate": "2022-12-18",
5   "FlightNumber": "{{Numero_de_vol}}",
6   "NumberOfTickets": "2"
7 }
```

The `"{{Numero_de_vol}}"` variable is highlighted with a red box and a red arrow pointing to it from the text above. The left sidebar shows a list of exercises, with **Prendre une reservation** selected under **Variables Globales**.

Utiliser la variable enregistrée

2^{ème} requête : Sur la requête **Prendre une reservation**, enregistrer le numéro de réservation

Sur l'onglet Tests, ajouter quelques snippets (tests) et personnaliser-les, comme suit

Exercice - Variables Globales / Prendre une reservation

POST `http://localhost:5000/flights_api/v1/resources/FlightOrders`

Params Authorization Headers (8) Body Pre-request Script Tests Settings

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag OrderNumber", function () {
6   pm.expect(pm.response.text()).to.include("OrderNumber");
7 });
8
9 pm.test("Le numéro de réservation est supérieur à 80", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData.OrderNumber).to.be.greaterThan(80);
12
13   //Ce commentaire pour la nouvelle variable globale
14   pm.globals.set("Num_de_reservation", jsonData.OrderNumber);
15
16 });
17
```

Snippets

Status code: Code is 200

Response body: Contains string
(la réponse doit contenir le numéro d'ordre)

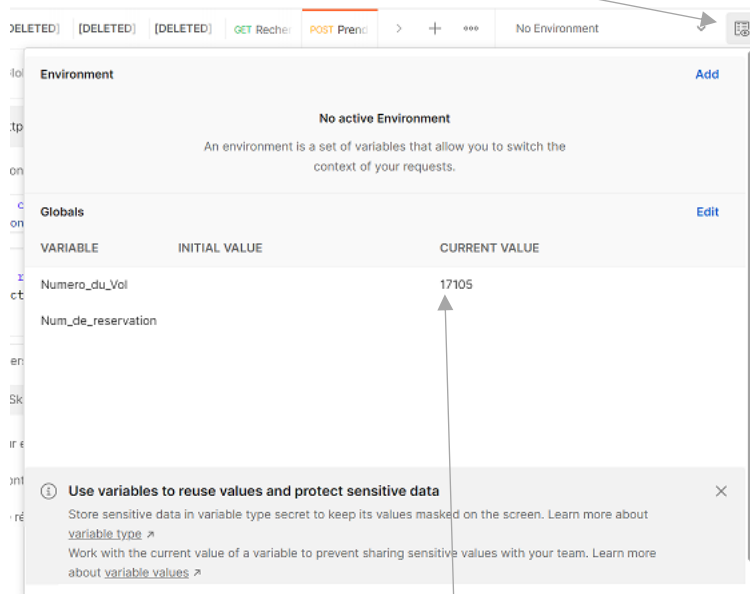
Response body: JSON value Check
Vérifier que le numéro d'ordre est plus grand que)

Set a global variable
(la nouvelle variable global est Num_de_reservation)

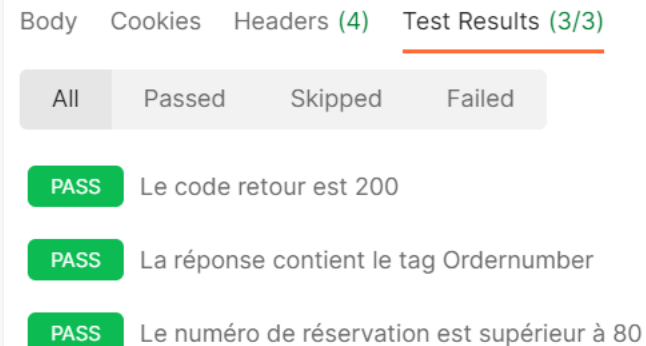
Utiliser la variable enregistrée

2^{ème} requête : Tester la requête *Prendre une reservation* seule

Cliquer sur l'icone Environnement
pour visualiser les variables globales



Lancer l'exécution de la requête



Vérifier la présence d'une valeur ou renseigner la (champ éditable)

Utiliser la variable enregistrée

3^{ème} requête : Prendre la requête **Rechercher une réservation par numéro** et ajouter la variable globale

Ajouter la variable globale dans l'url de la requête

The screenshot shows the Postman interface with the following details:

- Top Bar:** Home, Workspaces, API Network, Explore, Search Postman, Invite, Upgrade.
- Left Sidebar:** Collections (Exercise - Ajout de tests, Exercise - Requetes Simple, Exercise - Variables Globales), Environments, Mock Servers, Monitors.
- Request List:** [DELETED], [DELETED], [DELETED], GET Recher, POST Prend, GET Recl.
- Request Details:** GET, http://localhost:5000/flights_api/v1/resources/FlightOrders/{{Num_de_reservation}}
- Params:** Authorization, Headers (6), Body, Pre-request Script, Tests, Settings.
- Query Params Table:**

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Utiliser la variable enregistrée

Exécuter l'enchainement complet des tests

Sélectionner l'onglet **Runs** de la collection

Cliquer sur le bouton **Run Collection**

(Dé)Sélectionner les requêtes à exécuter

Cliquer sur le bouton **Run Exercice – Variables Globales**

Utiliser la variable enregistrée

Résultats de l'exécution

The screenshot displays the Postman interface with the 'Exercice - Variables Globales' collection selected. The 'Run results' tab is active, showing a summary of the test run. The summary table indicates that all tests passed, with a total duration of 404ms and 8 tests. The 'Console' tab is also visible, showing the raw logs for each test. An arrow points to the 'Console' tab with the text 'Cliquer sur l'onglet Console'. Another arrow points to the expanded log for the 'POST' test, with the text 'Déplier l'arborescence des logs pour voir les détails'.

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	none	1	404ms	8	20 ms

All Tests: Passed (8) Failed (0) Skipped (0)

View Summary

GET Recherche par ville de départ... http://localhost:5000/flights_api/v1/resources/Flights?Departur... / Recherc... 200 OK 13 ms 1.431 KB 1

Pass Le code retour est 200

Pass La réponse contient le tag FlightNumber

Pass Le premier vol est le 17105 et la variable globale 'Numero_de_vol' est créée

POST Prendre une reservation http://localhost:5000/flights_api/v1/resources/FlightOrders / Prendre une reservat... 200 OK 39 ms 187 B

Pass Le code retour est 200

Pass La réponse contient le tag Ordernumber

Pass Le numéro de réservation est supérieur à 80

GET Rechercher une réservation p... http://localhost:5000/flights_api/v1/resources/FlightOrders/{{Nu... / Recherc... 200 OK 8 ms 315 B

Pass Le code retour est 200

Pass OrderNumber est bien présent

Console

GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=2022-12-08 200 | 13 ms

POST http://localhost:5000/flights_api/v1/resources/FlightOrders%0A 200 | 39 ms

Network

Request Headers

Request Body

Response Headers

Response Body

GET http://localhost:5000/flights_api/v1/resources/FlightOrders/87 200 | 8 ms

Exercice 3 : Enchaîner plusieurs requêtes avec les variables globales

- 7. Exercice 3 : Enchaîner plusieurs requêtes avec les variables globales
 - 7.1. Enregistrer la donnée FlightNumber dans une variable globale
 - 7.2. Utiliser la variable dans la requête BookFlight et enregistrer le numéro de réservation
 - 7.3. Afficher le contenu des variables globales
 - 7.4. Exécuter l'enchaînement complet des tests

6

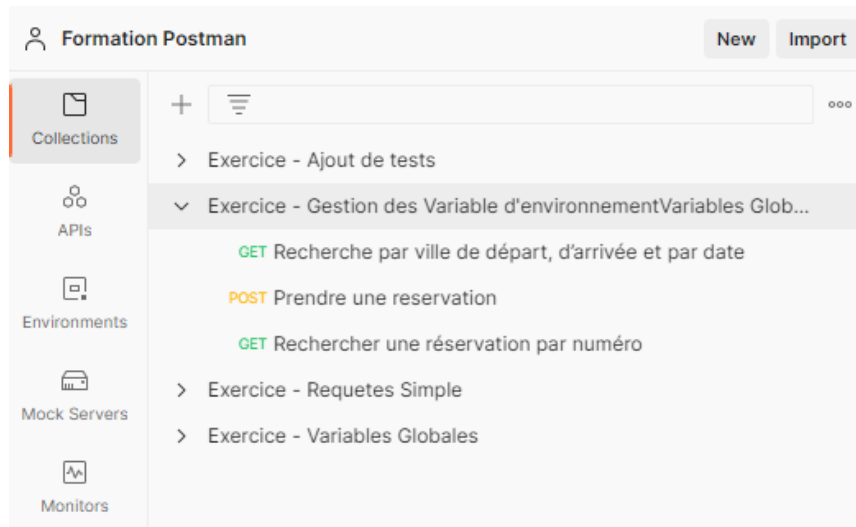
Utilisation des variables

<https://learning.postman.com/docs/sending-requests/variables/>

Gestion des variables d'environnement

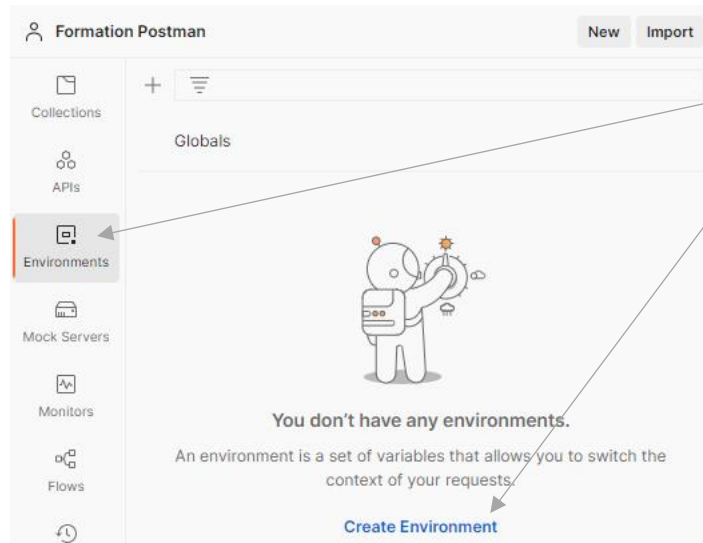
Objectif : Faciliter la maintenance des tests

Création d'une nouvelle collection en dupliquant la collection de l'exercice des variables Globales



Gestion des variables d'environnement

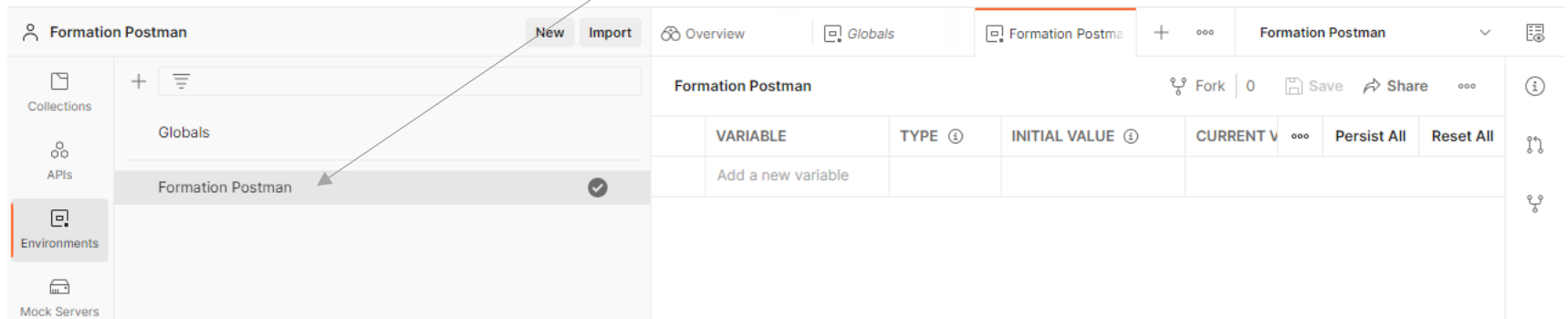
Création d'un environnement



Cliquer sur Environnement

Cliquer sur l'icone + ou sur le lien **Create Environment**

Renommer ce dossier pour le personnaliser



Gestion des variables d'environnement

Création de nouvelle variable

Cliquer sur le lien **Add a new variable**

Saisir son nom et sa valeur

Sauvegarder

The screenshot shows the Postman interface with the 'Environments' tab active. The 'Formation Postman' environment is selected. A table lists variables:

VARIABLE	TYPE	INITIAL VALUE	CURRENT VALUE
BaseURL	default	http://localhost:5000/fl...	http://localhost:5000/flights_api/v1/resour...
Date	default	2022-12-25	2022-12-25
Departure	default	Paris	Paris
Arrival	default	London	London
Add a new variable			

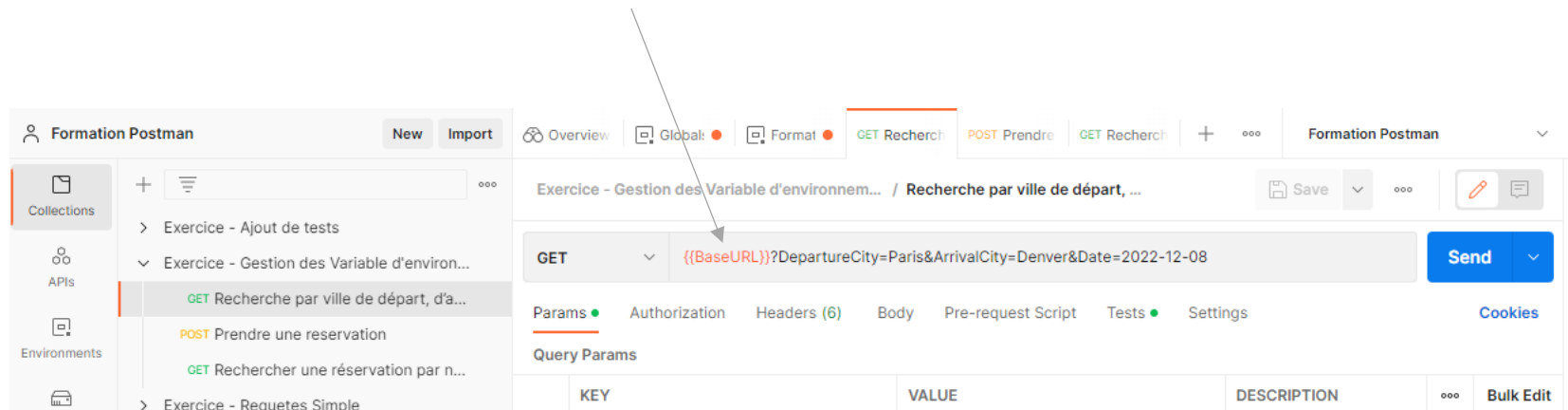
Vérifier que l'environnement est actif

The screenshot shows a context menu for the 'Formation Postman' environment. The menu includes options like 'Set as active environment', 'Move', 'Export', 'Duplicate', 'Create a fork', and 'Create pull request'. A message states 'This environment is already active.'

Gestion des variables d'environnement

Utilisation de la variable d'environnement

Remplacer http://localhost:5000/flights_api/v1/resources par `{{BaseURL}}` dans toutes les requêtes



En modifiant une seule donnée `{{BaseURL}}`,
les requêtes pointeront vers un nouvel environnement

Gestion des variables d'environnement

Utilisation d'une variable d'environnement

The screenshot shows a REST client interface with a sidebar on the left containing a tree view of exercises. The main panel displays a GET request to `{{BaseURL}}/Flights?DepartureCity={{&ArrivalCity=Denver&Date=2022-12-08}}`. Below the URL bar, the 'Query Params' tab is active, showing a table with the following data:

KEY	VALUE	DESCRIPTION
☒ DepartureCity	{{	
☒ ArrivalCity	G Numero_du_Vol	INITIAL
☒ Date	G Num_de_reservation	CURRENT 17105
	E BaseURL	SCOPE Global
	E Date	
	E Departure	
	E Arrival	

An arrow points from the text 'Saisir {{ , une liste des variables disponibles est proposée' to the dropdown menu in the 'VALUE' column.

Saisir {{ ,
une liste des variables
disponibles est proposée

The screenshot shows the same REST client interface with a POST request to `{{BaseURL}}/FlightOrders`. The 'Body' tab is active, showing a JSON payload:

```
1 {
2   "Class": "Economy",
3   "CustomerName": "bruno",
4   "DepartureDate": "{{Date}}",
5   "FlightNumber": "{{Numero_du_Vol}}",
6   "NumberOfTickets": "2"
7 }
```

An arrow points from the text 'Saisir les variables entre double {{ }}' to the variable placeholders in the JSON body.

Below the body editor, the 'Test Results' tab shows a successful response:

```
1 {
2   "OrderNumber": 92,
3   "TotalPrice": 278.94
4 }
```

Saisir les variables
entre double {{ }}

Gestion des variables d'environnement

Consultation des variables d'Environnement / Globales

Récupérer ici la liste des variables

The screenshot shows the Postman application interface. At the top, there is a search bar labeled 'Search Postman' and a blue 'Invite' button. Below the search bar, there is a row of tabs: '[CONFLI]', 'Form', 'GET Recl', 'POST Prend', 'GET Recher', and 'Exercic'. To the right of these tabs is a dropdown menu currently showing 'Formation Postman'. An arrow points from the text 'Récupérer ici la liste des variables' to this dropdown menu. Below the tabs, there is a section titled 'Globals' with a sub-header 'Global variables for a workspace edited by anyone in the workspace'. This section contains a table with columns 'VARIABLE' and 'VALUE'. The table lists three variables: 'Numero_du_Vol' with value '17105', 'Num_de_reservation' with value '92', and 'Add a new variable'. To the right of this table, there is a larger table titled 'Formation Postman' with columns 'VARIABLE', 'INITIAL VALUE', and 'CURRENT VALUE'. This table lists three variables: 'BaseUrl' with initial value 'http://localhost:5000/flights_api/v1/resources' and current value 'http://localhost:5000/flights_api/v1/resources', 'Date' with initial value '2022-12-25' and current value '2022-12-25', and 'VilleDepart' with initial value 'Paris' and current value 'Paris'. Below this table, there is another table titled 'Globals' with columns 'VARIABLE' and 'VALUE'. This table lists two variables: 'Numero_du_Vol' with value '17105' and 'Num_de_reservation' with value '92'.

VARIABLE	INITIAL VALUE	CURRENT VALUE
BaseUrl	http://localhost:5000/flights_api/v1/resources	http://localhost:5000/flights_api/v1/resources
Date	2022-12-25	2022-12-25
VilleDepart	Paris	Paris
VilleArrivee	London	London

VARIABLE	VALUE
Numero_du_Vol	17105
Num_de_reservation	92

Exercice 4 : Variables d'environnement

- 8. Exercice 4 : Variables d'environnement
 - 8.1. Création des variables d'environnement
 - 8.2. Rendre les requêtes indépendantes de l'adresse des services Rest
 - 8.3. Utiliser les variables ville de départ, ville d'arrivée et date dans GetFlights
 - 8.4. Modifier BookFlight en utilisant la variable Date
 - 8.5. Exécuter la séquence et visualiser les variables

Synthèse des variables

Instruction	Description
<code>pm.globals.set('myVariable', MY_VALUE);</code>	Sauvegarder dans variable globale
<code>pm.globals.get('myVariable');</code>	Lire une variable globale
<code>pm.globals.unset('myVariable');</code>	Réinitialiser un variable globale
<code>pm.globals.clear();</code>	Effacer toutes les variables globales
<code>pm.collectionVariables.set('myVariable', MY_VALUE);</code> <code>pm.collectionVariables.get('myVariable');</code> <code>pm.collectionVariables.unset('myVariable');</code> <code>pm.environment.clear();</code>	Idem pour les variables de collection
<code>pm.environment.set('myVariable', MY_VALUE);</code> <code>pm.environment.get('myVariable');</code> <code>pm.environment.unset("myVariable");</code> <code>pm.environment.clear();</code>	Idem pour les variables d'environnement
<code>pm.variables.get('myVariable');</code>	Idem pour les variables de requête
<code>pm.environment.name</code>	Lire le nom de l'environnement actif
<code>pm.iterationData.get('myVariable');</code>	Lire une donnée sur itération avec un fichier CSV/JSON
<code>pm.variables.replaceIn('{{ \$randomFirstName }}');</code>	Utiliser une variable dynamique
<code>console.log(myVar);</code>	Ecrire dans la log d'exécution

Synthèse des variables dynamiques

Postman dispose de fonctions de génération dynamique des données pour les tests

La liste complète des possibilités :

<https://learning.postman.com/docs/writing-scripts/script-references/variables-list/>

Variable dynamique	Description	Exempe
\$randomCity	City	East Ryanfurt, Jenkinsview
\$randomStreetName	Street name	Mckenna Pines, Schiller Highway, Vandervort Pike
\$randomStreetAddress	Street with number	98165 Tanya Passage, 0695 Monahan Squares
\$randomCountry	Country	Belgium, Antarctica (the territory South of 60 deg S)
\$randomCountryCode	Country code (2-letter)	GY, TK, BG
\$randomPrice	Price	244.00, 301.00
\$randomDatePast	Date in the past	Wed Mar 06 2019 04:17:52 GMT+0800 (WITA)
\$randomDateFuture	Date in the future	Wed Nov 20 2019 20:26:40 GMT+0800 (WITA)
\$randomDateRecent	Recent date	Thu Jun 20 2019 13:29:11 GMT+0800 (WITA)
\$randomCurrencyCode	Currency code	THB, HTG USD, AUD
\$randomCurrencyName	Currency name	Pound Sterling, Bulgarian Lev
\$randomPhrase	Phrase	We need to copy the online CSS microchip!
\$randomEmail	Email from popular email providers	Mable_Crist@hotmail.com, Ervin47@gmail.com
\$randomPassword	Password	v_Ptr4aTaBONsM0, 8xQM6pKgBUndK_J
\$randomLoremWord	Lorem ipsum word	ipsa, dolor, dicta
\$randomFirstName	First name	Dillan, Sedrick, Daniela
\$randomLastName	Last name	Schamberger, McCullough, Becker
\$randomPhoneNumber	Phone number	946.539.2542 x582, (681) 083-2162



7

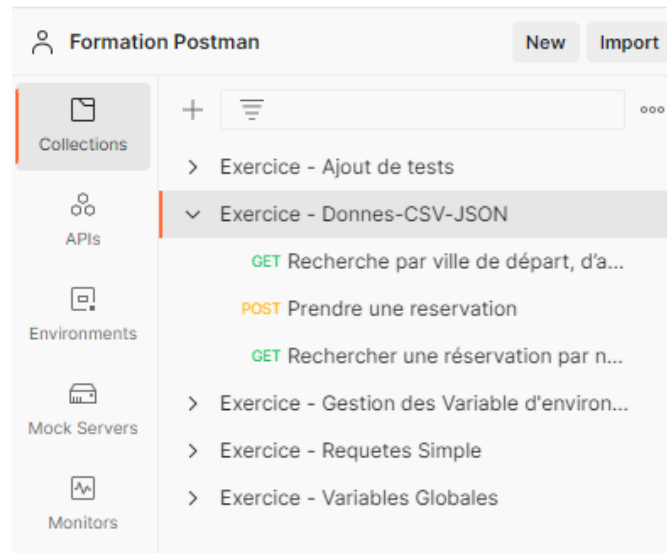
Utilisation des données CSV/JSON

<https://learning.postman.com/docs/running-collections/working-with-data-files/>

Utiliser les données dans un fichier CSV / JSON

Objectif : Automatiser une exécution d'une requête à partir de la lecture d'un fichier de données

Création d'une nouvelle collection en dupliquant la collection de l'exercice des variables d'environnement



Utiliser les données dans un fichier CSV / JSON

Préparation de la requête

Pour récupérer les données du fichier csv, la requête sera renseignée avec des variables portant le nom de la colonne à reprendre

	Flights_data.csv
1	Departure,Arrival,Date,FlightNumber,Price,SeatsAvailable
2	Seattle,Portland,2023-05-11,1225,138.47,250
3	London,San Francisco,2023-05-20,17022,171.8,250
4	Paris,Denver,2023-05-13,17040,163.8,250
5	Seattle,San Francisco,2023-05-18,1226,139.47,250
6	London,Zurich,2023-05-13,11038,159.8,250
7	Frankfurt,Paris,2023-05-07,11225,145.2,250
8	Paris,Los Angeles,2023-05-19,17122,156.47,250
9	Sydney,London,2023-05-12,11765,179.6,250
10	Seattle,Denver,2023-05-17,1212,125.47,250
11	San Francisco,Paris,2023-05-09,20221,112.2,250
12	

Exercice - Donnes-Csv-JS... / Recherche par ville de départ, d'arrivée et par d...

Save

GET

{{BaseURL}}/Flights?DepartureCity={{Departure}}&ArrivalCity={{Arrival}}&Date={{Date}}

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

	KEY	VALUE	DESCRIPTION		Bulk Edit
☒	DepartureCity	{{Departure}}			
☒	ArrivalCity	{{Arrival}}			
☒	Date	{{Date}}			
	Key	Value	Description		

Utiliser les données dans un fichier CSV / JSON

Exécution de la requête

Sélectionner la collection *Exercice – Données-CSV-JSON*

Sur l'onglet **Run**, cliquer sur le bouton **Run Collection**

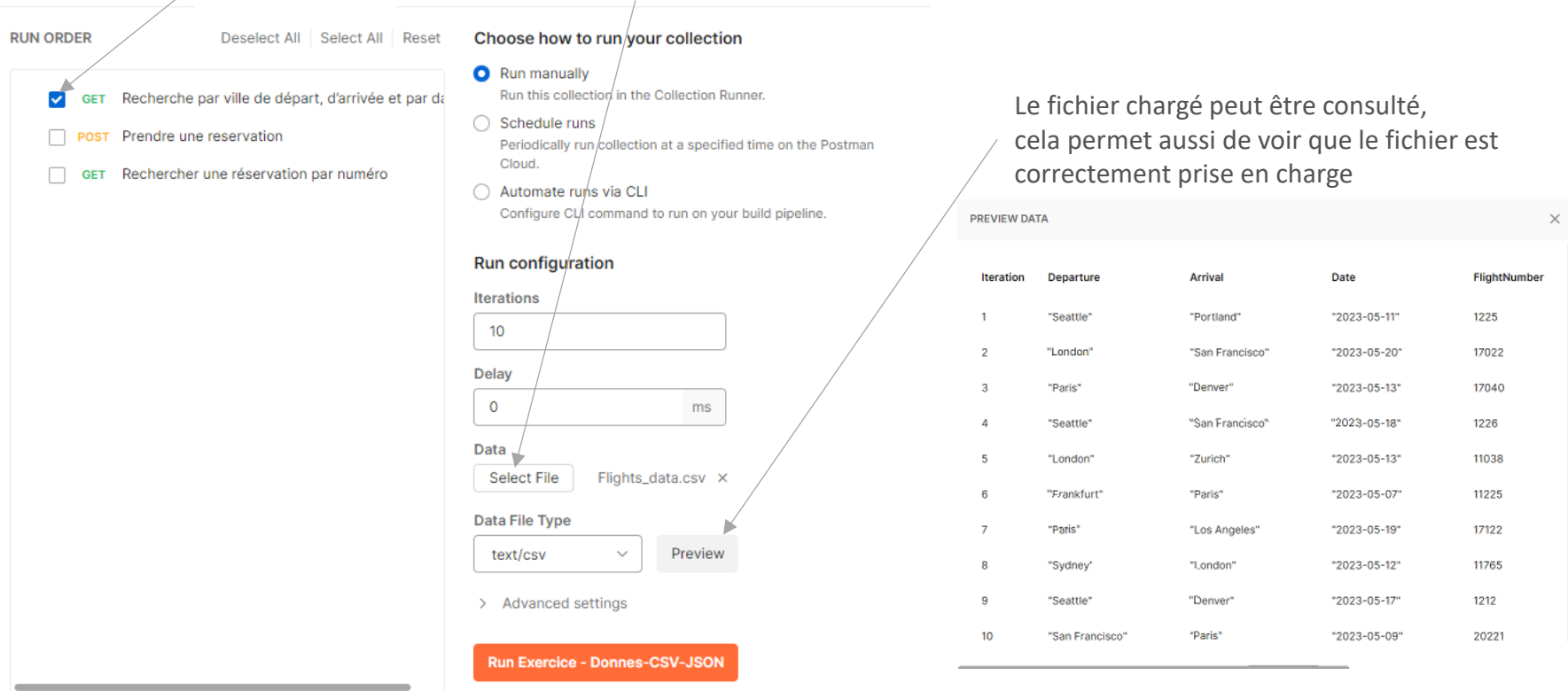
The screenshot displays the Postman interface. On the left sidebar, the collection 'Exercice - Donnes-CSV-JSON' is selected. The main panel shows the 'Runs' tab for this collection. At the top right of the main panel, there are links for 'Share', 'Fork', and a count of '0'. Below these, there are tabs for 'Authorization', 'Pre-request Script', 'Tests', 'Variables', and 'Runs', with 'Runs' being the active tab. Under the 'Runs' tab, there are two sub-tabs: 'Past runs' and 'Scheduled runs'. A message states: 'Runs triggered for this collection via Collection Runner and Postman CLI.' Below this, there are filters for 'Last 100 runs', 'Run by', 'Run status', and 'Source'. A table header is visible with columns: 'Start time', 'Source', 'Duration', 'All tests', 'Passed', 'Failed', and 'Skipped'. In the center of the table area, there is an illustration of an astronaut holding a clipboard, with the text 'Your collection has not been run yet' below it. At the bottom center, there is an orange button labeled 'Run Collection'.

Utiliser les données dans un fichier CSV / JSON

Exécution de la requête

Cocher la requête prête (la première)

Au niveau du champ **Data**, cliquer sur le bouton **Select File**



The screenshot shows the Postman interface for running a collection. The 'RUN ORDER' section on the left lists three requests, with the first one selected. The 'Choose how to run your collection' section in the center offers three options: 'Run manually' (selected), 'Schedule runs', and 'Automate runs via CLI'. Below this, the 'Run configuration' section allows setting 'Iterations' to 10 and 'Delay' to 0 ms. The 'Data' section shows a 'Select File' button and the file 'Flights_data.csv'. A 'Preview' button is also visible. The 'PREVIEW DATA' table on the right displays the data from the CSV file.

Iteration	Departure	Arrival	Date	FlightNumber
1	"Seattle"	"Portland"	"2023-05-11"	1225
2	"London"	"San Francisco"	"2023-05-20"	17022
3	"Paris"	"Denver"	"2023-05-13"	17040
4	"Seattle"	"San Francisco"	"2023-05-18"	1226
5	"London"	"Zurich"	"2023-05-13"	11038
6	"Frankfurt"	"Paris"	"2023-05-07"	11225
7	"Paris"	"Los Angeles"	"2023-05-19"	17122
8	"Sydney"	"London"	"2023-05-12"	11765
9	"Seattle"	"Denver"	"2023-05-17"	1212
10	"San Francisco"	"Paris"	"2023-05-09"	20221

Cliquer sur le bouton **Run Exercice – Donnee-CSV-JSON**

Utiliser les données dans un fichier CSV / JSON

Résultats de la requête

The screenshot displays the Formation Postman interface. On the left, the 'Collections' sidebar shows a tree structure with 'Exercice - Données-CSV-JSON' selected. The main panel shows the 'Run results' for this collection, including a table with columns: Source, Environment, Iterations, Duration, All tests, and Avg. Resp. Time. Below this, the 'All Tests' section shows 'Passed (10)', 'Failed (0)', and 'Skipped (0)'. The 'Iteration 9' and 'Iteration 10' sections show the test results for 'GET Recherche par ville de départ, d...'. The 'Console' tab at the bottom shows the response body for the last request, which is a JSON array of flight data.

Deux dernières exécutions

Détail de la réponse de la dernière requête

```
[{"Airline": "AA", "ArrivalCity": "Paris", "ArrivalTime": "02:23 PM", "DepartureCity": "San Francisco", "DepartureTime": "07:12 AM", "FlightNumber": 20210, "Price": 112.2, "SeatsAvailable": 250, "DayOfWeek": "Tuesday"}, {"Airline": "AA", "ArrivalCity": "Paris", "ArrivalTime": "06:23 PM", "DepartureCity": "San Francisco", "DepartureTime": "11:12 AM", "FlightNumber": 20221, "Price": 112.2, "SeatsAvailable": 250, "DayOfWeek": "Tuesday"}]
```

Utiliser les données dans un fichier CSV / JSON

Exécuter une séquence de test (avec deux requêtes)

The screenshot shows a REST client interface with a sidebar on the left containing a list of exercises. The main area displays a GET request to `{{BaseURL}}/Flights?DepartureCity={{Departure}}&ArrivalCity={{Arrival}}&Date={{Date}}`. Below the request, the 'Tests' tab is active, showing a sequence of three tests. The first test checks the status code is 200. The second test checks that the response text includes the tag 'FlightNumber'. The third test uses `pm.iterationData.get("FlightNumber")` to retrieve data from a CSV file and expects the response text to include it. An arrow points from this code in the tests to a CSV file named 'Flights_data.csv' shown in a separate window below.

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("le numero de vol attendu dans la réponse " + pm.iterationData.get("FlightNumber"), function
10  () {
11   pm.expect(pm.response.text()).to.include(pm.iterationData.get("FlightNumber"));
12 });
```

Sur la requête *Recherche par ville de départ*

Ajouter la fonction **`pm.iterationData.get`** (« ») pour récupérer les données du fichier csv (utiliser le même nom)

Et pour pouvoir faire une comparaison avec les résultats de la réponse

The screenshot shows a CSV file named 'Flights_data.csv' with the following data:

1	Departure,Arrival,Date,FlightNumber,Price,SeatsAvailable
2	Seattle,Portland,2023-05-11,1225,138.47,250
3	London,San Francisco,2023-05-20,17022,171.8,250
4	Paris,Denver,2023-05-13,17040,163.8,250

Utiliser les données dans un fichier CSV / JSON

La deuxième requête, récupère **{{FlightNumber}}** du fichier CSV

The screenshot displays a REST client interface with a sidebar on the left containing a list of exercises. The main area shows two requests for the exercise 'Prendre une reservation'.

Request 1:

- Method: POST
- URL: `{{BaseURL}}/FlightOrders`
- Body (JSON):

```
1 {
2   ... "Class" : "Economy", ...
3   ... "CustomerName" : "bruno", ...
4   ... "DepartureDate" : "{{Date}}", ...
5   ... "FlightNumber" : "{{FlightNumber}}", ...
6   ... "NumberOfTickets" : "2"
7 }
```

Request 2:

- Method: POST
- URL: `{{BaseURL}}/FlightOrders`
- Tests (Script):

```
1 pm.test("Vérifier le prix total", function () {
2   var jsonData = pm.response.json();
3   prix = pm.iterationData.get("Price")
4   pm.expect(jsonData.TotalPrice).to.eql(2*prix);
5 });
```

Ici, vérification que

le prix total (dans la réponse) = Nb de ticket * Prix (fichier CSV)

La fonction **`pm.iterationData.get`** (« ») est utilisée pour récupérer le prix des billets

Utiliser les données dans un fichier csv et JSON

Lancer l'enchainement des requêtes (voir slides 55 & 56)

The screenshot displays the OnePoint API testing interface. On the left, a sidebar lists test exercises, with 'Exercice - Donnees-CSV-JSON' selected. The main area shows the 'Run results' for this exercise, including a summary table and a detailed view of the test execution.

Exercice - Donnees-CSV-JSON - Run results

Run on Today, 15:35:57 · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	Formation Postman	10	1s 727ms	40	17 ms

All Tests Passed (40) Failed (0) Skipped (0) [View Summary](#)

POST Prendre une reservation `{{BaseURL}}/FlightOrders` / Prendre une reservation **200 OK** 37 ms 188 B

Pass Vérifier le prix total

Iteration 10

GET Recherche par ville de départ, d'ar... `{{BaseURL}}/Flights?DepartureCity={{Departure}}&ArrivalCit...` / Recherc... **200 OK** 6 ms 585 B

Pass Le code retour est 200

Pass La réponse contient le tag FlightNumber

Pass le numero de vol attendu dans la réponse 20221

POST Prendre une reservation `{{BaseURL}}/FlightOrders` / Prendre une reservation **200 OK** 15 ms 187 B

Pass Vérifier le prix total

At the bottom of the interface, there are tabs for 'Find and Replace', 'Console', 'Cookies', 'Capture requests', 'Runner', 'Trash', and a search icon.

Exercice 5 : Utiliser les données dans un fichier csv et JSON

- 9. Exercices 5 : Utiliser les données dans un fichier csv et JSON
 - 9.1. Utiliser les données dans un fichier csv
 - 9.2. Exécuter la séquence de test
 - 9.3. Utiliser les données dans un fichier json



8

Création d'une boucle

<https://learning.postman.com/docs/running-collections/building-workflows/>

Création d'une boucle

Objectif : Répéter la même séquence de tests

Création d'une nouvelle collection avec les requêtes suivantes :

Rechercher une réservation par le nom du client

GET {{BaseURL}}/FlightOrders?CustomerName={{customer}}

Supprimer une réservation par numéro

DELETE {{BaseURL}}/FlightOrders/{order}

✓ Exercice - Créer une boucle

GET Rechercher une réservation par le nom du client

DEL Supprimer une reservation par numéro

Création d'une boucle

Objectif :

Récupérer la liste des réservations

Supprimer une réservation

Répéter la boucle si le nombre de réservation est supérieur à 3

+

☰

...

> Exercice - Ajout de tests

> Exercice - Créer une boucle

GET Rechercher une réservation par le nom du ...

DEL Supprimer une reservation par numéro

> Exercice - Donnees-CSV-JSON

> Exercice - Gestion des Variable d'environnement

> Exercice - Requetes Simple

> Exercice - Variables Globales

Exercice - Créer une bo... / Rechercher une réservation par le nom du c...

Save

...

✎

📄

GET

⌵

{{BaseURL}}/FlightOrders?CustomerName=bruno

Send

⌵

Params

Auth

Headers (6)

Body

Pre-req.

Tests

Settings

Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	CustomerName	bruno			

Body

Cookies

Headers (4)

Test Results (1/1)

200 OK

8 ms

10.59 KB

Save Response

Pretty

Raw

Preview

Visualize

📄

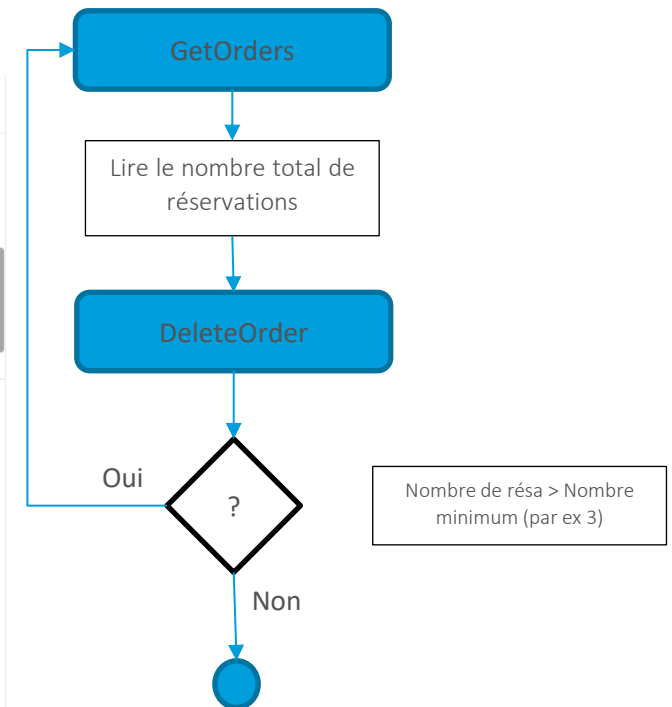
🔍

```

[{"Class": "First", "CustomerName": "bruno", "DepartureDate": "2022-12-18 18:15", "FlightNumber": 15113, "NumberOfTickets": 5, "OrderNumber": 82, "TotalPrice": 277.23}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 83, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 84, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 85, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 86, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 87, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 88, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 89, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 90, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 91, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 92, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 93, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 94, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 95, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 96, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 97, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 98, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 99, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 100, "TotalPrice": 278.94}

```

Liste des réservations



Création d'une boucle

Préparation de la première requête : Récupérer la liste des réservations

The screenshot shows the Postman interface for a GET request. The URL is `{{BaseURL}}/FlightOrders?CustomerName=bruno`. The 'Tests' tab is active, displaying a JavaScript test script. The script performs three main actions: 1. Verifies the client name from the first reservation. 2. Verifies that at least one reservation exists. 3. Verifies that the first reservation's order number is greater than 80 and saves it in a global variable.

```
1 //Vérification du nom du client de la première reservation
2 pm.test("Vérifier le nom du client ", function () {
3     var jsonData = pm.response.json();
4     console.log(jsonData.length)
5     pm.expect(jsonData.length).to.be.gt(1)
6
7     pm.globals.set("Nb_Reservation", jsonData.length);
8 });
9
10 //Récupérer le nombre de reservation
11 pm.test("Vérifier qu'il existe au moins une réservation", function () {
12     var jsonData = pm.response.json()
13     pm.expect(jsonData[0].CustomerName).to.be.eql("bruno");
14 });
15
16 //Récupérer les numéro de la première reservation
17 pm.test("Vérifier que le numéro de réservation est supérieur à 80", function () {
18     var jsonData = pm.response.json();
19     pm.expect(jsonData[0].OrderNumber).to.be.greaterThan(80)
20
21     //Sauvegarde de la 1er reservation dans une variable globale Num_Reservation
22     pm.globals.set("Num_Reservation", jsonData[0].OrderNumber);
23 });
24
```

Informations récupérées après exécution:

Identifiant de la première réservation
Le nombre de réservation total

Globals		
VARIABLE	INITIAL VALUE	CURRENT VALUE
Num_Reservation		82
Nb_Reservation		63

Où au niveau des logs

The screenshot shows the Postman console with the 'Console' tab selected. It displays the GET request to `http://localhost:5000/flights_api/v1/resources/FlightOrders?CustomerName=bruno` and the response body, which is the number 63.

Création d'une boucle

Préparation de la deuxième requête : Supprimer une réservation

Récupération de la variable globale du numéro de réservation à supprimer

The screenshot shows the Postman interface with a collection named 'Exercice - Créer une boucle'. Inside this collection, there is a sub-collection 'Exercice - Supprimer une réservation par numéro'. The selected request is a DELETE request to the endpoint `{{BaseURL}}/FlightOrders/{{Num_Reservation}}`. The response is a 200 OK status with a body containing `"true": "Order deleted 82"`.

Request Configuration:

- Method: DELETE
- URL: `{{BaseURL}}/FlightOrders/{{Num_Reservation}}`
- Query Params:

KEY	VALUE	DESCRIPTION
Key	Value	Description

Response:

200 OK 39 ms 174 B

Body: `"true": "Order deleted 82"`

Console:

```
DELETE http://localhost:5000/flights_api/v1/resources/FlightOrders/82
```

Création d'une boucle

Préparation de la deuxième requête : Réaliser la boucle

Ajouter sur l'onglet **Pre-request Script** de la requête **Supprimer une réservation** :

1- Affichage du nombre total de réservation dans la console. Utiliser le snippet **Get global variable**

2- La boucle est réalisée avec la commande **postman.setNextRequest ("")**;

Avec le nom de la requête qui doit suivre : **Rechercher une réservation par le nom du client**

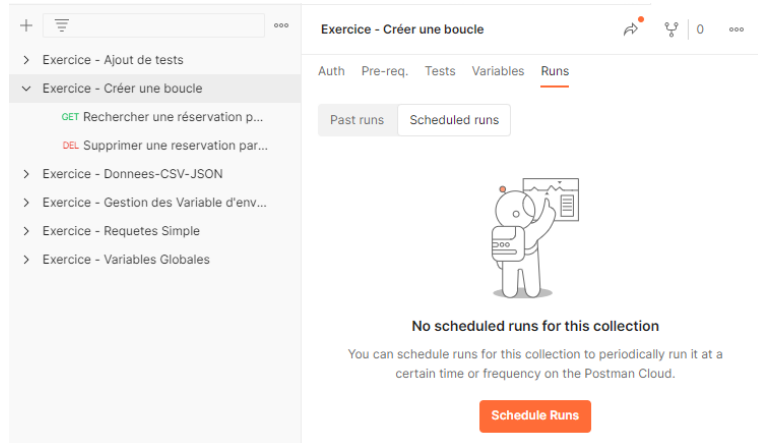
3- Ajouter une condition pour permettre cet enchainement (if condition { }).

The screenshot shows the Postman interface. On the left, a sidebar lists several exercises, with 'Exercice - Créer une boucle' expanded to show two requests: 'GET Rechercher une réservation p...' and 'DEL Supprimer une reservation par...'. The main panel displays the 'DEL Supprimer une reservation par...' request. The URL is '{{BaseURL}}/FlightOrders/{{Num_Reservation}}'. The 'Pre-request Script' tab is active, showing the following JavaScript code:

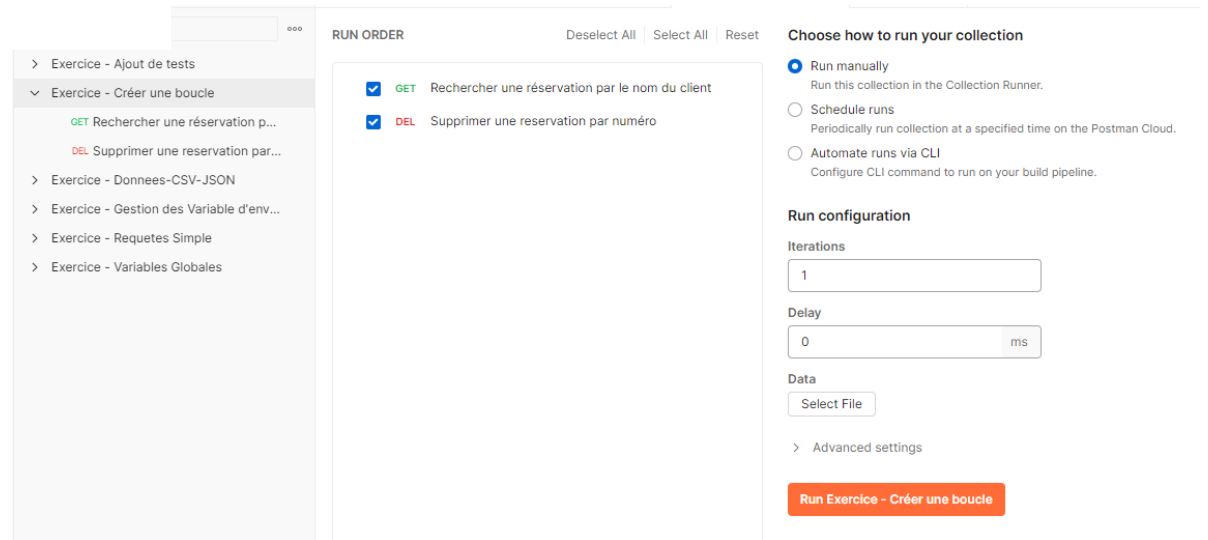
```
1 var Compteur_Reservation = pm.globals.get("Nb_Reservation");
2 console.log ("le nombre de reservation est : " + Compteur_Reservation);
3
4 if (Compteur_Reservation > 2) {
5   postman.setNextRequest ("Rechercher une réservation par le nom du client");
6 }
7
```

Création d'une boucle

Lancement de la boucle



Sur l'onglet **Runs**,
Cliquer sur le bouton **Schedule Runs**



Choisir **Run Manually**

Cliquer sur le bouton **Run Exercice – Créer une boucle**

Création d'une boucle

Résultats

Enchaînement des tests

Décompte au niveau
de la console

Exercice - Créer une boucle - Run results

Run on Today, 12:01:36 · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	Formation Postman	1	9s 355ms	204	17 ms

All Tests Passed (204) Failed (0) Skipped (0) [View Summary](#)

- Pass Vérifier que le numéro de réservation est supérieur à 80
- DELETE Supprimer une réservation par numéro {{BaseURL}}/flightsOrders/{{Num_Reservation}} / Supprimer une r... 200 OK 13 ms 175 B
- Pass Le code retour est 200
- GET Rechercher une réservation par le nom du client {{BaseURL}}/flightsOrders?CustomerName=bruno / Recherch... 200 OK 6 ms 487 B
- Pass Vérifier le nom du client
- Pass Vérifier qu'il existe au moins une réservation
- Pass Vérifier que le numéro de réservation est supérieur à 80
- DELETE Supprimer une réservation par numéro {{BaseURL}}/flightsOrders/{{Num_Reservation}} / Supprimer une r... 200 OK 11 ms 175 B
- Pass Le code retour est 200

Console

```
"le nombre de reservation est : 4"
DELETE http://localhost:8080/flights_api/v1/resources/FlightOrders/141 200 29 ms
GET http://localhost:8080/flights_api/v1/resources/FlightOrders?CustomerName=bruno 200 6 ms
3
"le nombre de reservation est : 3"
DELETE http://localhost:8080/flights_api/v1/resources/FlightOrders/142 200 13 ms
GET http://localhost:8080/flights_api/v1/resources/FlightOrders?CustomerName=bruno 200 6 ms
2
"le nombre de reservation est : 2"
DELETE http://localhost:8080/flights_api/v1/resources/FlightOrders/143 200 11 ms
```

Exercice 6 : Création d'une boucle

- 6. Exercice 6 : Créer une boucle
- 6.1. Lire le nombre total de réservation
- 6.2. Réaliser la boucle



9

API Mock / API Monitor

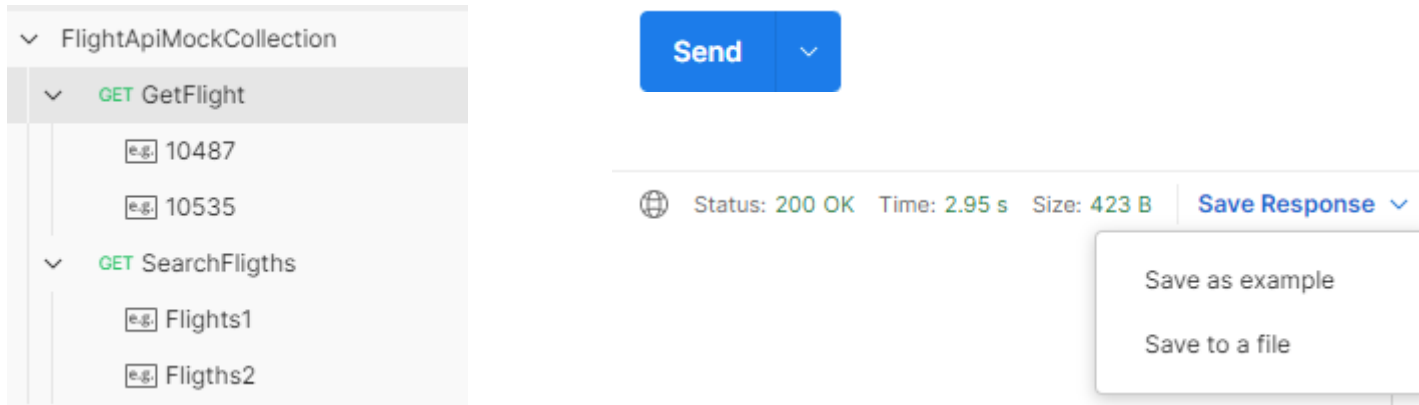
<https://learning.postman.com/docs/designing-and-developing-your-api/mocking-data/setting-up-mock/>

Utilité du Mock API

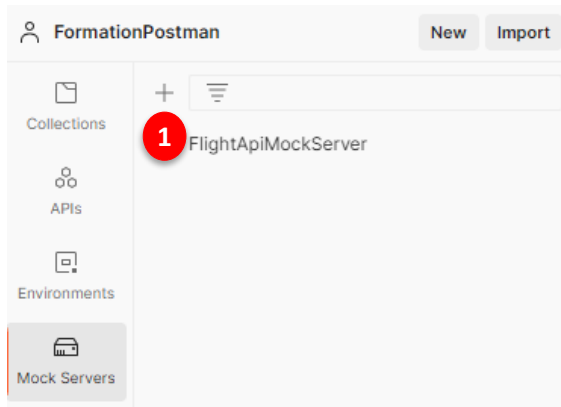
- Permet de simuler le fonctionnement réel d'un API (bouchon).
- Modifier manuellement la réponse pour anticiper une future évolution de l'API.
- Rendre les tests indépendants d'une API qui n'est pas disponible.
- Economiser le prix des APIs payantes en phase de test

Processus de création d'un serveur Mock

- Créer une collection comportant les requêtes à simuler.
- Créer un serveur Mock à partir de la collection
- Créer autant d'exemples que nécessaire pour simuler la requête
- Utiliser les requêtes simulées dans vos tests



Création d'un serveur Mock



Create a mock server

1. Select collection to mock 2. Configuration

Create a new collection Select an existing collection 2

Select a collection you want to mock.

Search collection

FlightApi_TNR

FlightApi

FlightApiMockCollection

Create a mock server

✓ Select collection to mock 2. Configuration

Mock Server Name

FlightApiMockServer2

Collection

FlightApiMockCollection

Environment

An environment is a group of variables useful for storing and reusing values.

[Learn more](#)

No Environment

☐ Save the mock server URL as an new environment variable

This will create a new environment containing URL.

Simulate a fixed network delay

No delay selected

Back

Create Mock Server

3

FlightApiMockServer2

Copy URL View Collection Docs Edit Configuration

Search your calls...

Refresh Logs



No mock server calls yet

To call the mock server, follow these steps:

1. Send a request to the following mock server URL, followed by the request path:

<https://fff534a22-3cbd-4b01-8e67-84d9c0594639.mock.pstmn.io>

2. Add examples responses to each request that the mock server will return.

[Learn what examples are and how to use them](#)

4

onepoint.

Utiliser les requêtes simulées

- La variable url est créée automatiquement dans un environnement FlightApiMockServer

- Exemples d'appels

{{url}}/Flights/10535

{{url}}/Flights?DepartureCity=Paris&ArrivalCity=London&Date=2022-12-30

TestMockServer / Flights_10535    Save

GET  {{url}}/Flights/10535

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Query Params

	KEY	VALUE	DESCRIPTION
	Key	Value	Description

Body Cookies Headers (13) Test Results  200 OK 256 m

Pretty Raw Preview Visualize JSON  

```
1 {  
2   "Airline": "AF",  
3   "ArrivalCity": "London",  
4   "ArrivalTime": "10:30 AM",  
5   "DepartureCity": "Paris",
```

  Online  Find and Replace  Console

▶ GET https://816fbe85-bcc1-4ee8-837d-502ed1d2c961.mock.pstmn.io/Flights/10535

Processus de création d'un monitor

- L'application à surveiller doit être accessible sur internet
- Créer une collection comportant les requêtes à surveiller.
- Créer un Monitor à partir de la collection
- Programmer la fréquence de surveillance
- Lancer la surveillance

Les requêtes utilisées pour le monitor

https://api.taiga.io/api/v1/projects/by_slug?slug=himhah-demo_api

```
pm.test("Project name is Demo_Api ", function () {  
  var jsonData = pm.response.json();  
  pm.expect(jsonData.name).to.eql("Demo_Api");  
  pm.globals.set("ProjectId", jsonData.id);  
});
```

<https://api.taiga.io/api/v1/issues?project={{ProjectId}}>

```
pm.test("Issues count is greater than 0", function () {  
  var jsonData = pm.response.json();  
  pm.expect(jsonData.length).to.greaterThan(0);  
  pm.globals.set("IssueId", jsonData[0].id);  
});
```

<https://api.taiga.io/api/v1/issues/{{IssueId}}>

Processus de création d'un monitor

1

Collections

APIs

Environments

Mock Servers

Monitors

Flows

You have no monitors set up

A monitor lets you run a collection periodically to check for its performance and response.

Create a Monitor

Create a monitor

Monitor name

TaigaApiMonitor

3

Collection

Taiga_Monitor

Collection tag

Choose a collection tag for this monitor. If the collection is not linked to an API, a monitor can only be created on the current tag.

CURRENT

Environment

The variable values in this environment will be used in all your monitoring calls. [Learn more](#)

None

Data file (optional) ⓘ

Only JSON and CSV files are accepted. Max 1 MB. [Learn more](#)

Select File

Run this monitor ⓘ

Check your usage limits

Week timer

Every day

3

3:00 PM

Create Monitor

4

This monitor hasn't run yet

When it does, the results will appear here. You can either wait for it to run as per its schedule or trigger a run manually.

Run

5

Résultat de la surveillance

Le résultat est aussi consultable sur internet avec votre compte postman <https://identity.getpostman.com/login>

TaigaApiMonitor **HEALTHY**

 Run  Refresh

View: ☒ Run summary ☐ Individual requests

Filter By:

All Requests



Type: All



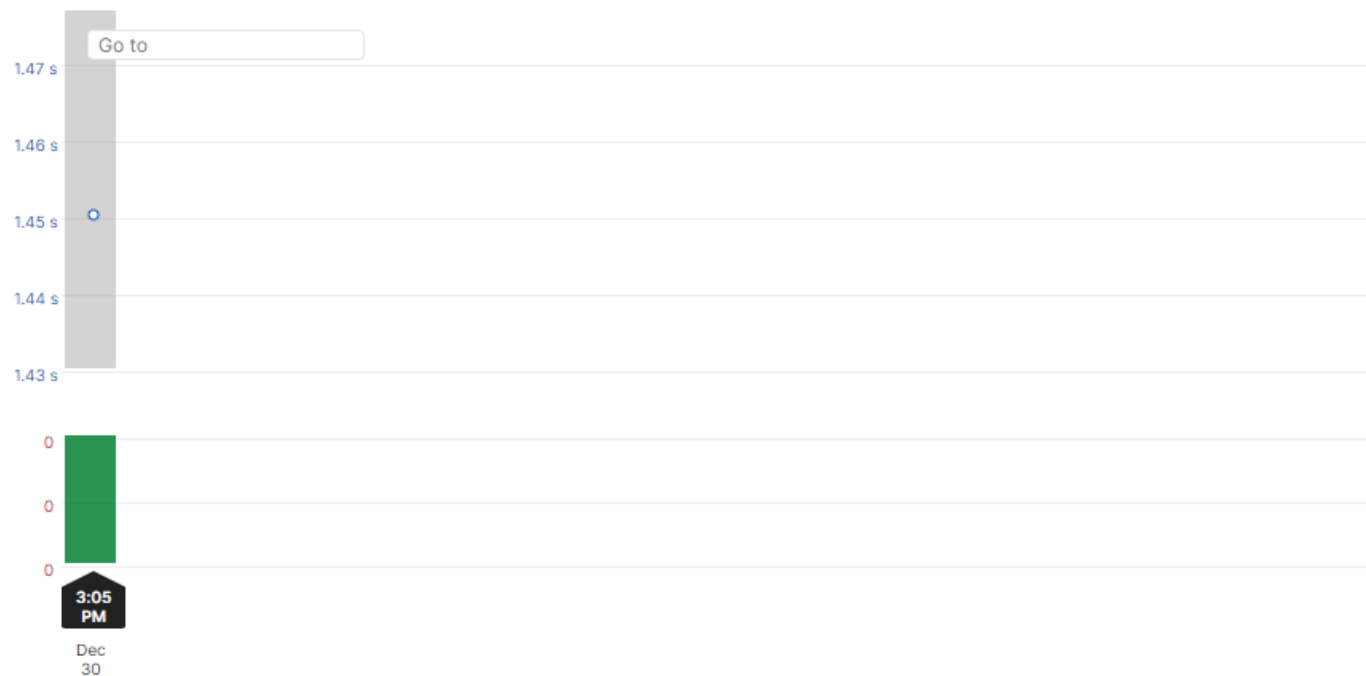
Run result: All



US (East)



Clear Filters





10

Documentation des APIs

<https://swagger.io/docs/specification/about/>

Standard OpenAPI

Standard indépendant du langage de programmation.

Une panoplie très riche d'utilitaires.

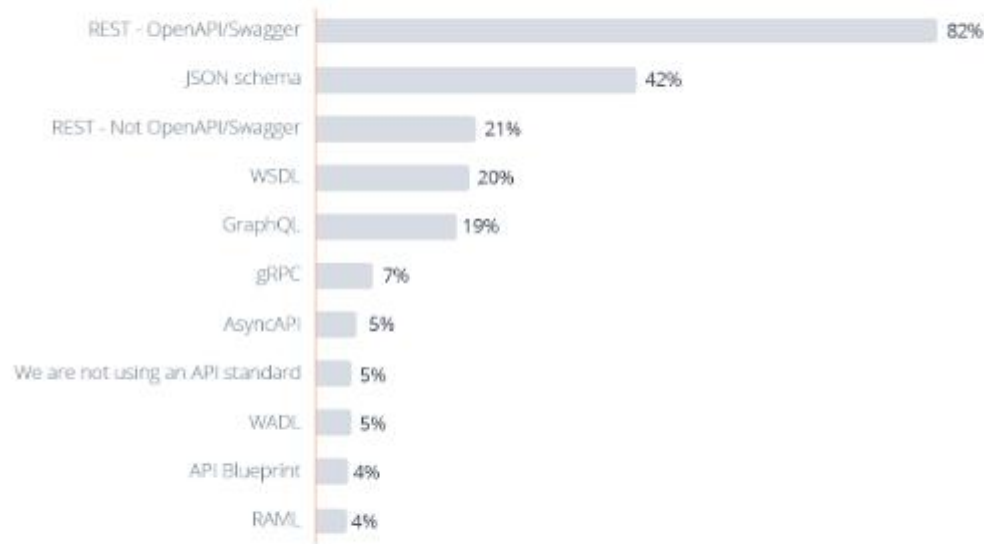
Format lisible et facilement interprétable.

Avantages :

- Validation de la description (Syntaxe, version de la spécification, directives de formatage).
- Validation des données.
- Génération de documentation.
- Génération de code.
- Éditeurs graphiques.
- Gestion des bouchons (Mock)
- Analyse de la sécurité

Un standard populaire

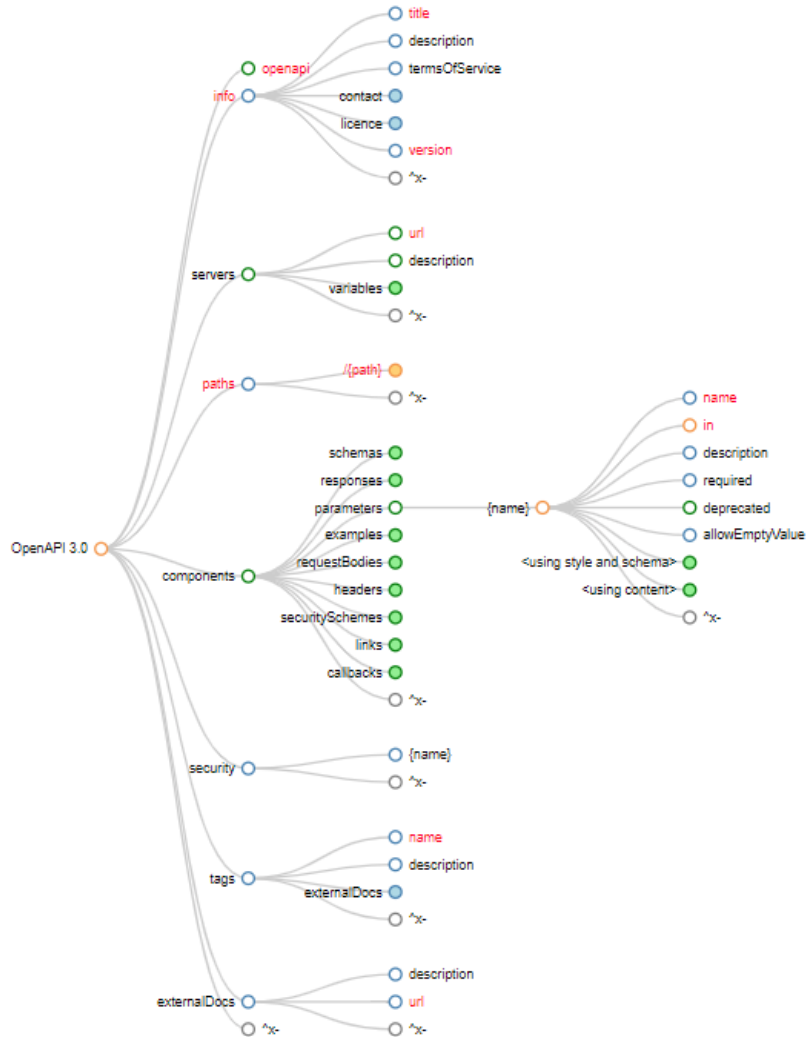
API Design



82% of organizations use the OpenAPI Specification (formerly known as Swagger) in their API development, up from 69% last year

Source : <https://smartbear.com/resources/ebooks/the-state-of-api-2020-report/>

Format de la documentation OpenAPI



Source : <https://openapi-map.apihandyman.io/>

Exemples de documentation openapi

Recherche de la documentation publique sur <https://app.swaggerhub.com/search>

Exemple à explorer :

- <https://app.swaggerhub.com/apis/LizaPolishchuk/SalonsBE/1.0.0#>
- <https://app.swaggerhub.com/apis/DeOliveiraJim/ShopManager/2.0#/ShopAns>

<https://www.postman.com/postman/workspace/postman-open-technologies-convert-postman-collections-into-openapi/overview>

Générer la documentation à partir de Postman

Exporter la collection (format JSON)

Installer postman-to-openapi (nécessite node.js)

```
npm i postman-to-openapi -g
```

Convertir la collection Postman

```
p2o ./path/to/PostmantoCollection.json -f ./path/to/result.yml -o ./path/to/options.json
```

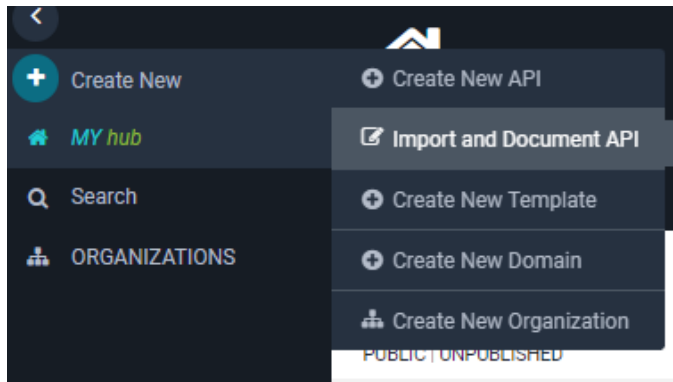
```
S:\github\PyFlightApi\tests>p2o ./FlightApi_TNR.postman.json -f ../Doc/PyFlightsApi-Dec22.yaml
openapi: 3.0.0
info:
  title: FlightApi_TNR
  version: 1.0.0
servers:
  - url: http://localhost:5000
  - url: http://{{endpoint}}
components:
  securitySchemes:
    basicAuth:
      type: http
      scheme: basic
tags:
  - name: Authentication
  - name: Cities
  - name: Flights
  - name: FlightOrders
paths:
  /login:
    get:
```

<https://www.postman.com/postman/workspace/postman-open-technologies-convert-postman-collections-into-openapi/overview>

Importer la documentation dans swagger.io

Créer un compte sur <https://app.swaggerhub.com/>

Importer la documentation générée

A screenshot of the 'Import API' form in the SwaggerHub web interface. The form has a title 'Import API' and a close button. Below the title, it says 'Enter path or URL to your API JSON or YAML spec, or browse to select a local file.' There are three input fields: 'Path or URL' with the value 'PyFlightsApi-Dec22.yaml', 'Owner' with the value 'imhah-hassan', and 'Project' with the value '— None —'. There is also a 'Visibility' dropdown set to 'Public'. At the bottom, there are two buttons: 'CANCEL' and 'UPLOAD FILE'.A screenshot of the 'Import API' form in the SwaggerHub web interface, showing a success message. The form has a title 'Import API' and a close button. Below the title, it says 'Update the name and version below, or keep the values from the spec.' There are two input fields: 'Name' with the value 'PyFlighAPI' and 'Version' with the value 'Dec22'. Below these fields, there is a green success message: 'Success. The resource is valid YAML'. At the bottom, there are two buttons: 'CANCEL' and 'IMPORT DEFINITION'.

Editer la documentation avec swagger.io

The screenshot displays the Swagger UI editor interface for a project named "PyFlighAPI" with a version of "Dec22". The interface is divided into several sections:

- Top Bar:** Contains navigation buttons: "Show Navigation", "Hide Code Editor", "Show Visual Editor", and "Hide UI Docs".
- Left Sidebar:** A navigation menu with sections for "Info", "Tags", "Servers", and "Search". Below these are expandable sections for "Cities", "Flights", and "FlightOrders", each listing various API endpoints and their methods (e.g., GET, POST, DELETE, PATCH).
- Central Editor:** A code editor showing the OpenAPI specification in YAML format. The code includes details about the API version (3.0.0), title (FlightApi_TNR), version (Dec22), servers (localhost:5000), security schemes (basicAuth), and specific endpoints like /login, /cities, and /cities/cmn.
- Right Panel:** A visual representation of the API documentation. It shows the title "FlightApi_TNR" with version tags "Dec22" and "OAS3". Below this, there's a "Servers" section with a dropdown menu and an "Authorize" button. The main part of the right panel lists the API endpoints grouped by category: "Authentication" (with a /login endpoint), "Cities" (with /cities and /cities/cmn endpoints), and "Flights" (with /login, /cities, and /cities/cmn endpoints). Each endpoint is color-coded by its method (blue for GET, green for POST/PATCH, red for DELETE).

At the bottom of the interface, there's a status bar showing "Last Saved: 1:24:21 pm - Dec 29, 2022" and a "VALID" indicator.

Annexes

- Exercices complémentaires
- Exécution en ligne de commande
- Intégration Jenkins
- Exercices complémentaires avec taiga.io
- Présentation de l'application de démo
- Liens utiles

Exercices complémentaires

1. Prendre une réservation sur une ville de départ et une ville d'arrivée aléatoirement choisies dans une liste de villes (utiliser GET http://localhost:5000/flights_api/v1/resources/Cities)
2. Prendre une réservation sur un vol choisi au hasard (utiliser GET http://localhost:5000/flights_api/v1/resources/Flights?Date=2023-02-08)
3. Prendre une réservation avec une date aléatoire dans le futur.
4. Lire toutes les réservations prises et vérifier le prix de chaque réservation

Exercices complémentaires avec taiga.io

- Documentation de l'API taiga.io

<https://docs.taiga.io/api.html>

- Ressources utilisées

<https://docs.taiga.io/api.html#auth-normal-login>

<https://docs.taiga.io/api.html#projects-list>

<https://docs.taiga.io/api.html#issues-list>

- Projet de démonstration (créer un nouveau projet)

https://tree.taiga.io/project/himhah-demo_api/issues

- Exercices

1. Lire les informations du projet à partir de son nom (slug = himhah-demo_api)
2. Lire la liste des tickets
3. Afficher les détails d'un ticket
4. S'authentifier et utiliser le token récupérer dans la suite des requêtes (header **Authorization: Bearer** {{token}})
5. Mettre à jour les informations d'un ticket
6. Créer un ticket (utiliser le paramétrage Issue types, severities, priorities)
7. Créer plusieurs tickets à partir d'un fichier Json
8. Supprimer tous les tickets créés.

Exécution en ligne de commande

<https://learning.postman.com/docs/postman-cli/postman-cli-overview/>

```
powershell.exe -NoProfile -InputFormat None -ExecutionPolicy AllSigned -Command  
"[System.Net.ServicePointManager]::SecurityProtocol = 3072; iex ((New-Object System.Net.WebClient).DownloadString('https://dl-  
cli.pstmn.io/install/win64.ps1')) »
```

Installation dans %USERPROFILE%\AppData\Local\Microsoft\WindowsApps

Exporter la collection à exécuter et copier le fichier .json dans ce répertoire

Exécuter la collection :

```
postman collection run DemoCLI.postman_collection.json
```

```
DemoCLI  
→ project  
GET https://api.taiga.io/api/v1/projects/by_slug?slug=himhah-demo_api [200 OK, 2.89kB]  
→ issues  
GET https://api.taiga.io/api/v1/issues?project=404046 [200 OK, 2.89kB]  
→ issueByRef  
GET https://api.taiga.io/api/v1/issues?project=404046 [200 OK, 2.89kB]  
→ authentication  
POST https://api.taiga.io/api/v1/auth [200 OK, 2.67kB, 479ms]
```

	executed	failed
iterations	1	0
requests	4	0
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0

```
total run duration: 2s  
total data received: 19.3kB (approx)  
average response time: 439ms [min: 101ms, max: 1012ms, s.d.: 360ms]
```

Postman api key is required for publishing run details to postman cloud.
Please specify it by using the postman login command

Intégration Jenkins

Installer newman (node.js nécessaire)

```
npm install -g newman
```

Exécuter la collection avec newman (exporter la collection depuis Postman)

```
newman run DemoCLI.postman_collection.json
```

Installer Jenkins avec les plugins **Junit**, **Test Results Analyzer**, **Git** et **Purge Job History**

Configurer un « Job » Jenkins

- **Gestion de code source : Git**

- Repository URL : <https://github.com/imhah-hassan/PyFlightApi.git>
- Branch : */REST_0323

- **Construire périodiquement :**

H/2 * * * *

- **Exécuter une ligne de commande batch Windows :**

```
SET PATH=C:\Apps\Python310;C:\Apps\Python310\Scripts;%PATH%
pip install PyJWT requests Flask python-dateutil
start python flight_rest.py
```

- **Exécuter une ligne de commande batch Windows :**

```
SET PATH=C:\Users\h.imhah\AppData\Roaming\npm;%PATH%
newman run tests\jenkins\JenkinsDemo.json --disable-unicode --reporters cli,junit
```

- **Publier le rapport des résultats des tests Junit**

```
newman/*.xml
```

Résultats Jenkins

Options Download Test (CSV) Search: Test/Class/Package

Chart	Package/Class/Testmethod	Passed	Transitions	59	58	57	56	55	54	53	52	51	50
☐	● (root)	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	● DemoCli	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	Body matches auth_token string	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	Check project name	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	Status code is 200	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	Verify first issue subject	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	Verify issue status name	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	Verify issues count greather than 0	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED
☐	Verify user name	100% (100%)	0	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED

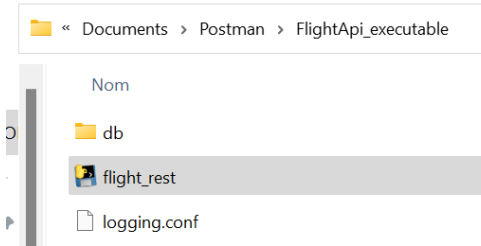
Top 10 Most Broken Tests

Installation de FlightApi

- Récupérer l'archive

<https://onepoint365.sharepoint.com/:u:/r/sites/TestIT-Communaute/Documents%20partages/Tutorials/Postman/FlightApi/FlightApi.zip>

- Décompresser la dans un répertoire de travail



- Lancer l'exécutable *flight_rest*

```
C:\Users\l.lescaut\OneDrive - x + v
FlightsApp Rest Api
Listening on http://localhost:5000
Sample request : http://localhost:5000/flights_api/v1/resources/Flights/10487
* Serving Flask app "flight_rest" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
```

- Vérifier le bon fonctionnement de l'application
 - Copie la requête de l'écran précédent (http://localhost:5000/flights_api/v1/resources/Flights/10487), puis
 - Coller la dans la barre d'adresse d'un navigateur



Liste des requêtes FlightApi utilisées

URL : http://localhost:5000/flights_api/v1/resources

Documentation : <https://app.swaggerhub.com/apis/imhah-hassan/PyFlightsApi/Dec22>

Nom de la requête	Ressource	Méthode	Paramètre
Rechercher un vol par son numéro	/Flights/{FlightNumber}	GET	Aucun
Recherche un vol par ville de départ, d'arrivée et par date	/Flights	GET	DepartureCity=Paris&ArrivalCity=Denver&Date=2021-12-08
Rechercher une réservation par le nom du client	/FlightOrders	GET	CustomerName={{customer}}
Rechercher une réservation par numéro	/FlightOrders/{order}	GET	Aucun
Supprimer une réservation par numéro	/FlightOrders/{order}	DELETE	Aucun
Supprimer toutes les réservations	/FlightOrders	DELETE	Aucun

Nom de la requête	Ressource	Méthode	Headers	Body
Prendre une réservation	/FlightOrders	POST	Content-Type : application/json	{ "Class" : "Economy", "CustomerName" : "IMHAH", "DepartureDate" : "2022-12-08", "FlightNumber" : "1060", "NumberOfTickets" : "2" }
Modifier une réservation par numéro	/FlightOrders/{order}	PATCH	Content-Type : application/json	{ "Class" : "First", "CustomerName" : "IMHAH", "DepartureDate" : "2021-12-09", "FlightNumber" : "1060", "NumberOfTickets" : "5" }

Liste des données de FlightApi utilisées (extrait)

FlightNumber ▼	DepartureInitials ▼	Departure ↕	DayOfWeek ↕	ArrivalInitials ▼	Arrival ↕	DepartureTime ↕	ArrivalTime ▼	Airline ▼	SeatsAvailable ▼
10535	PAR	Paris	Friday	LON	London	8:00 AM	10:30 AM	AF	250
10538	PAR	Paris	Friday	LON	London	10:24 AM	11:24 AM	AF	250
10539	PAR	Paris	Friday	LON	London	10:24 AM	12:54 PM	AF	250
10543	PAR	Paris	Friday	LON	London	12:48 PM	3:18 PM	AF	250
10547	PAR	Paris	Friday	LON	London	3:12 PM	5:42 PM	AF	250
11120	PAR	Paris	Friday	LON	London	3:50 PM	5:31 PM	NW	250
11774	PAR	Paris	Friday	LON	London	9:50 AM	11:31 AM	NW	250
13119	PAR	Paris	Friday	LON	London	6:14 PM	7:55 PM	NW	250

Liens utiles

- Documentation complète

<https://learning.postman.com/docs/getting-started/introduction/>

- Documentation rapide

<https://postman-quick-reference-guide.readthedocs.io/en/latest/index.html>

- La référence de javascript en Postman :

<https://learning.postman.com/docs/writing-scripts/script-references/postman-sandbox-api-reference/>

- Téléchargement : <https://www.postman.com/downloads/>

- <https://blog.nicolashachet.com/developpement-php/larchitecture-rest-expliquee-en-5-regles/https://www.tutorialspoint.com/soapui>

- <https://www.eewee.fr/postman-cest-quoi>

- Pour comprendre la syntaxe Postman,

https://developer.mozilla.org/fr/docs/Learn/Getting_started_with_the_web/JavaScript_basics